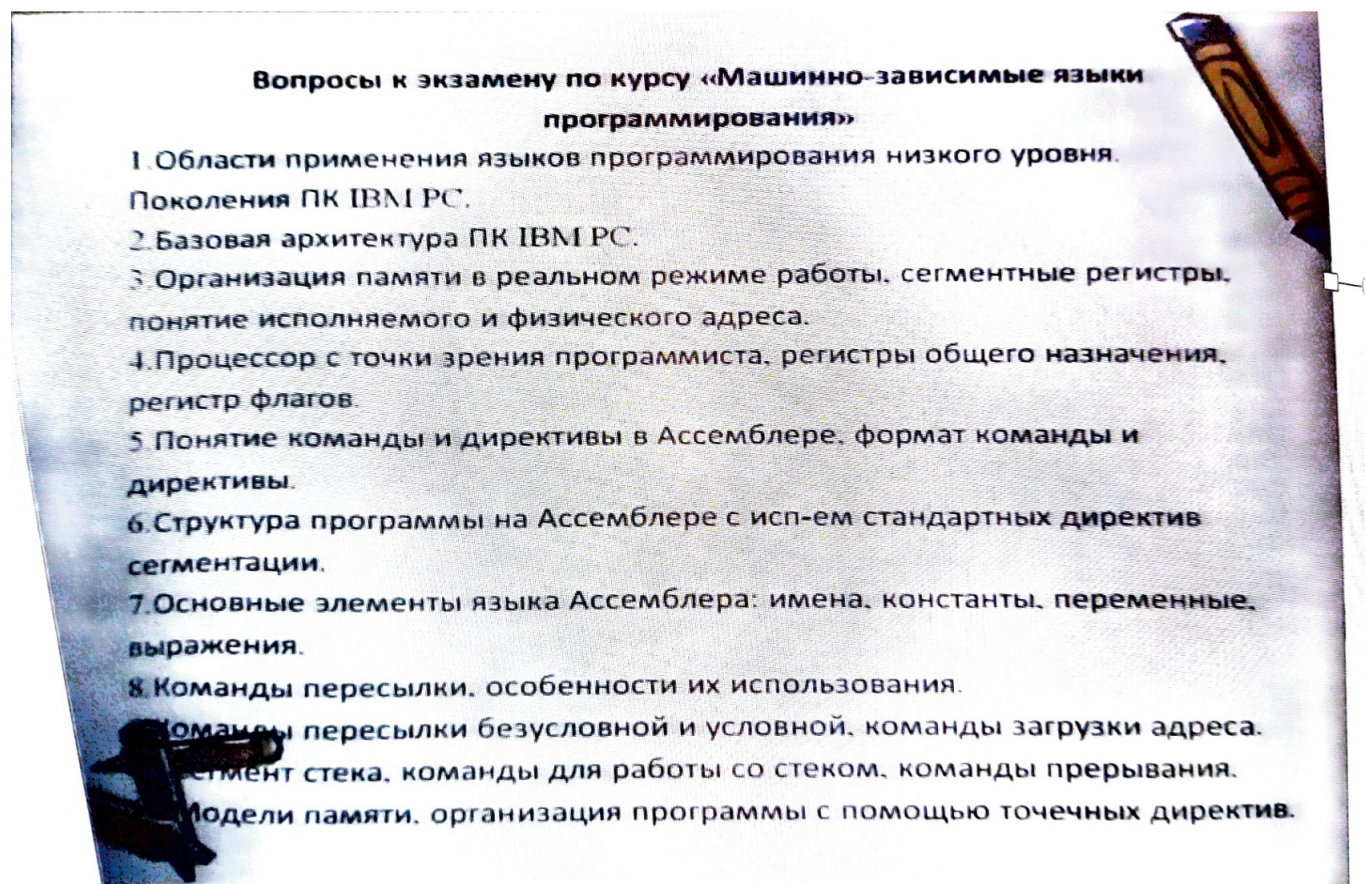


Материал к экзамену
(Переработка презентаций Федоровой А. Г. с исправлениями и пояснениями)
Создано при помощи Lua^AT_EX 1.15.0

Первый слайд



1. Области применения языков программирования низкого уровня. Поколения IBM PC.

У машинно-ориентированных языков программирования две сферы применения:

- разработка системных программ, включаемых в состав ОС, например, драйверы устройств;
- решение специализированных задач информационных и управляющих систем, к которым относят **СУБД** и **системы управления языком интерфейса, бортовые** программы сбора и обработки информации сложных технических устройств.

Поколения IBM PC:

1. IBM PC — 1981;
2. IBM PC AT (Advanced Technology) — 1984;
3. 32-разрядный i386 процессор — 1987;

4. i486 процессор — 1990;
5. 64-разрядный микропроцессор Pentium — 1993;
6. Pentium Pro, Pentium 2, Pentium 3 — ?;
7. Pentium 4 (кодовое назв. «Willamate») — 2002.

2. Базовая архитектура IBM PC.

В современных ПК реализован **магистрально-модульный** принцип построения. Все устройства (модули) подключены к центральной магистрали, **системной шине**, которая включает в себя адресную шину, шину данных и шину управления.

Шина — это набор линий связи, по которым передается информация от одного из источников к одному или нескольким приемникам. *Адресная шина* однонаправленная, адреса передаются от процессора. *Шина данных* двунаправленная, данные передаются как от процессора, так и к процессору. В *шину управления* входят линии связи и однонаправленные и двунаправленные.

Для синхронизации работы процессора и внешних устройств в ПК также включены система прямого доступа к памяти (DMA), а также интерфейсные блоки.

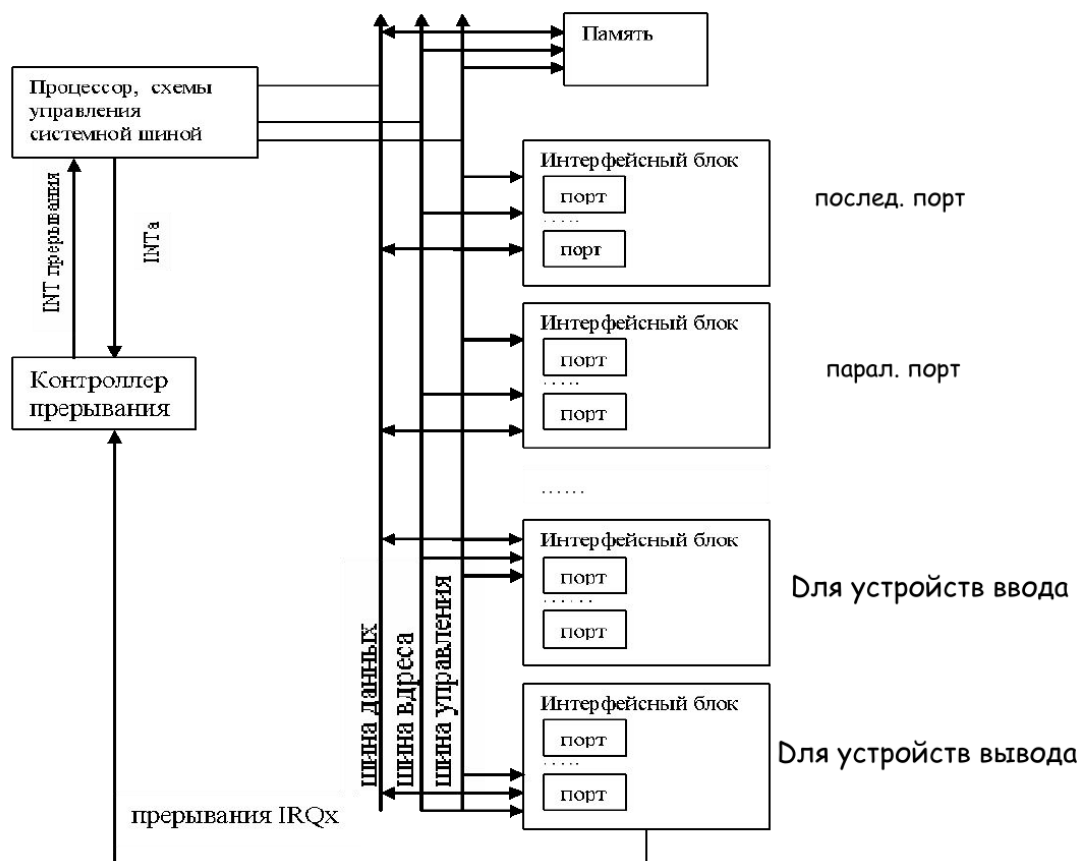


Рис. 1: Архитектура ПК

3. Организация памяти в реальном режиме работы, сегментные регистры, понятие исполняемого и физического адреса.

Процессор *ix86* после включения питания устанавливается в реальный режим адресации памяти и работы процессора. Большинство ОС сразу переводит его в защищенный режим.

Процессор может работать с памятью по-разному: с моделью *flat* или с использованием **сегментов**.

В реальном режиме размер сегмента фиксирован и равен 64 Кбайта. Адрес сегмента кратен 16 и в 16-ричной с. с. может быть записан в виде $XXXX0_{16}$. Старшие 4 цифры могут быть записаны в один из сегментных регистров:

- DS, ES, FS, GS — для определения начала сегментов данных;
- CS — начало кодового сегмента;
- SS — начало сегмента стека.

Они также называются **селекторами**.

Физический адрес байта записывается как

сегмент:смещение

и может быть получен по формуле

$$\text{ФА} = \text{АС} + \text{ИА}.$$

- АС — **адрес сегмента** (значение из сегментного регистра $\ll 4$);
- ИС — **исполняемый адрес** (смещение, в байтах, относительно начала сегмента).

4. Процессор с точки зрения программиста, регистры общего назначения, регистр флагов.

Совокупность программно доступных средств процессора называется **архитектурой процессора**, с точки зрения программиста.

Начиная с *i386* процессора программисту доступны 16 **основных регистров**, 11 **регистров для работы с сопроцессором и мультимедийными приложениями**, и в реальном режиме доступны некоторые регистры управления и некоторые специальные регистры.

Регистр — набор из n устройств (**триггеров**) способный поразрядно хранить двоичное число.

Регистры в *i386*:

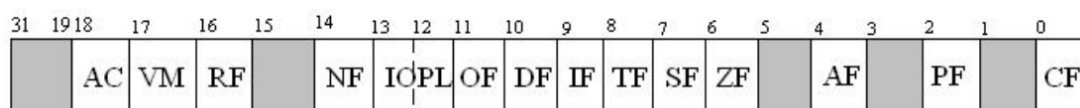
- Общего назначения: EAX, EBX, ECX, EDX;
- Регистры индексов и указателей: ESI, EDI, ESP, EBP;
- Сегментные регистры (см. выше);
- Счетчик (указатель) команд — EIP;
- Регистр флагов — EFLAGS.

Регистры общего назначения могут использоваться для временного хранения адресов и данных. При работе с 16-разрядными данными к ним можно обращаться как AX, BX, CX, DX, а при работе с байтами — к старшей (верхней) и младшей (нижней) части 16-разрядного регистра, например, AH и AL.

Регистры общего назначения имеют названия, связанные с их функцией:

- AX — аккумулятор (накопление);
- BX — база (адресация операндов по базе);
- CX — счетчик (организация циклов);
- DX — регистр данных;

Также в процессоре есть **регистр флагов** — EFLAGS. Он определяет состояние процессора и программы в каждый момент времени:



- CF - перенос
- PF - четность
- AF - полуперенос
- ZF - флаг нуля
- SF - флаг знака
- TF - флаг трассировки
- IF - флаг прерывания
- DF - флаг направления
- OF - флаг переполнения
- AC - флаг выравнивания операндов
- VM - флаг виртуальных машин
- RF - флаг маскирования прерывания
- NT - флаг вложенной задачи
- IOPL - уровень привилегий ввода/вывода.

- CF — перенос (беззнаковое переполнение); перенос за разрядную сетку при сложении, заем для старшего разряда при вычитании. Пример для 4-битных регистров:

$$1111_2 + 0001_2 = 0000_2, CF = 1$$

$$0000_2 - 0001_2 = 0000_2 + 1111_2 = 1111_2, CF = 1$$

- PF — четность; 1, если в младшем байте результата содержится четное число единиц;
- AF — переполнение половины байта (*Auxiliary Flag*); устанавливается в 1, если при сложении происходит перенос из 3-го разряда в 4-й или если при вычитании происходит заем из 4-го разряда в 3-й.
- ZF — флаг нуля; устанавливается в 1, если результат равен 0.
- SF — флаг знака; равен знаковому разряду результата.
- TF — трассировка; прерывать работу процессора после каждой команды.
- DF — обработка строк; если DF = 0, то обработка идет в направлении увеличения адресов, если DF = 1, то обработка в направлении уменьшения адресов.
- OF — знаковое переполнение; 1, если в результате знаковой операции произошло переполнение. То есть если у обоих операндов знаковые биты равны *одному значению*, а после выполнения операции результат имеет *другое значение* знакового бита. Пример для 4-битных регистров:

$$0100_2 + 0100_2 = 1000_2, OF = 1$$

$$1000_2 + 1000_2 = 0000_2, OF = 1$$

- IOPR — если уровень привелегий для текущей программы меньше или равен значению IOPR, то программе **разрешены** операции ввода/вывода (инструкции in и out). На 8086 и i186 этот флаг всегда равен 1.
- NT — режим работы вложенных задач.
- RF — маскирование некоторых прерываний процессора.
- VM — защищенный режим или режим виртуальной машины.
- AC — если 1 и адреса операндов длиной в слово или двойное слово не кратны 2 и 4 соответственно, происходит ошибка.

Замечание: при выполнении беззнаковых операций, флаг OF не дает никакой полезной информации, только CF. Аналогично, при выполнении знаковых операций, при проверке на ошибки, имеет значение только флаг OF.

Еще одно: непонятный флаг IOPL по идее контролирует выполнения операций ввода/вывода на кольцах ОС (*RINGS*). То есть если IOPL = 2, то коду на кольцах 0, 1 и 2 будет разрешено выполнять ввод и вывод. Получается, что в презентации (по крайней в той версии, которая у меня на руках) ошибка.

5. Понятие команды и директивы в Ассемблере, формат команды и директивы.

[<метка>[:]] <код операции> [<операнды>, ...] [; комментарии]

Команда — это цифровой двоичный код, состоящий из двух подпоследовательностей двоичных цифр, одна из которых определяет код операции (действие), вторая — операнды, участвующие в операции, место хранения результата. Занимает в памяти от 1 до 15 байт.

x86 работает с безадресными, одноадресными, двухадресными и трехадресными командами.

Операнды могут находиться в регистрах, памяти или быть константными выражениями.

Директива — псевдооперация — информирует Ассемблер о чем-либо, например, какой объем памяти выделить под переменную или как следует объединять инструкции для формирования программного модуля, но сама директива в собранной программе никак не проявляется.

Директива состоит из четырех полей:

[<симв. имя>] <код псевдооперации> <операнды> [; комментарии]

Операндов может быть любое количество.

6. Структура программы на Ассемблере с использованием стандартных директив сегментации.

Исходный модуль просматривается Ассемблером, пока не встретится директива `end`. Обычно программа на Ассемблере состоит из трех сегментов: стек, данные и код.

```
1 ; сегмент стека
2 Sseg segment stack 'stack'
3     db 256 dup(?)
4     ; ...
5 Sseg ends
6
```

```

7  ; сегмент данных
8  Dseg segment 'data'
9      ; ...
10 Dseg ends
11
12 ; сегмент кода
13 Cseg segment 'code'
14     assume CS:Cseg, DS:Dseg, SS:Sseg ; связь сегментных регистров
15                                     ; и сегментов
16                                     ; Это позволяет Ассемблеру правильно
17                                     ; добавить префиксы при обращении к
18                                     ; регистрам. Например, SP
19                                     ; эквивалентен SS:SP, а SS = Sseg.
20 Start proc far ; главная процедура
21     ; ...
22     ret
23 Start endp
24 Cseg ends
25
26 end Start

```

7. Основные элементы языка Ассемблер: имена, константы, переменные, выражения.

Символические имена в Ассемблере могут состоять из строчных и прописных букв латинского алфавита, цифр от 0 до 9 и символов `_`.

В программе на Ассемблере могут использоваться **константы** пяти типов: целые двоичные (`11000011b`), десятичные (`125`), шестнадцатеричные (`21h`, `0ABh`), действительные с плавающей точкой (`0.547e-2`), символьные (`'A'`).

Константам можно присваивать символические имена при помощи директив `equ` и `=`:

```

M equ 27
B = 21h

```

Строковые литералы: `'abc123'`, `"hello"`.

Переменные объявляются при помощи директивы *определения* `dX`, где *X* зависит от размера переменной:

- `byte` — 1 байт — `db`;
- `word` — 2 байта — `dw`;
- `dword` — 4 байта — `dd`;

- ...

Пример:

```
A dw 24000
CRLF db 0Ah, 0Dh
```

У этой директивы также существует другой вариант, упрощающий выделение памяти под массивы данных:

```
Arr db 20 dup(0) ; выделить место под 20 байт,
                  ; заполнить память нулями
```

Если память ничем инициализировать не нужно, можно использовать специальный символ `?`.

Существуют следующие операции:

- арифметические: `+`, `-`, `*`, `/`, `mod`;
- логические: `not`, `and`, `or`, `xor`;
- сравнения: `lt`, `le`, `eq`, `ne`, `gt`, `ge`;
- сдвига: `shl`, `shr`;
- `offset <имя>` — смещение операнда внутри сегмента.

Привычные с виду арифметические выражения на языке Ассемблера могут быть только **константными выражениями**. Это значит, что все операнды выражения должны быть определены на этапе ассемблирования программы. Например, допустимыми выражениями будут

```
1000100b + 37,
M lt B,
27 shl 3.
```

8. Команды пересылки, особенности их использования.

Команды пересылки позволяют копировать данные из регистра в регистр, из памяти в регистр и из регистра в память. Но у них есть свои особенности использования:

1. Нельзя переслать информацию из памяти в память.

2. Нельзя переслать информацию из одного сегментного регистра в другой.
3. Нельзя переслать константное выражение в сегментный регистр напрямую.
4. Значение в CS не может быть изменено командой mov.
5. Байты в памяти хранятся в прямом порядке, а в регистрах — в обратном.

Да, конечно, некорректно говорить, что в регистрах байты хранятся «в обратном порядке». Здесь, точно так же, как и в памяти, байты хранятся в порядке увеличения их номеров. Просто память обычно изображается как вещь, растущая справа налево, а регистр — как вещь, имеющая младшие байты справа, а старшие слева.

```

; ...
R dw 1234h ; в байте с адресом R -- 34h,
           ; в байте с адресом (R + 1) -- 12h
; ...
mov AX, R   ; AH = 12h, AL = 34h

```

6. Размер передаваемых данных определяется типом операндов в команде.

Это важно, поскольку команда `mov [SI], 0` вызовет ошибку сборки. В этом случае надо использовать оператор

<тип> ptr <выражение>

Выражение — константное или адресное, тип — размер области памяти: `byte`, `word`, `dword`, `qword`.

Тогда следующие команды будут допустимыми и эквивалентными:

`mov byte ptr [SI], 0`
 или `mov [SI], byte ptr 0`.

7. В командах с двумя операндами, тип одного операнда должен совпадать с типом другого.

9. Команды пересылки безусловной и условной, команды загрузки адреса.

К командам пересылки также относятся:

- Команды обмена — `xchg R, R/M`;
- Команда перестановки байт (i386 и новее) — `bswap R32`;

- Команды конвертирования — `cbw (AL → AX)`, `cwd (AX → DX : AX)`, `cwde (AX → EAX)`, `cdf (EAX → EDX : EAX, f=Федорова, потому нигде в Интернете упоминания этой команды нет)`;
- Команды условной пересылки — `cmovXX R, R/M`, где `XX` — операция сравнения. Если условие верно — переслать.
- Загрузка адреса — `lea R, M` — вычислить адрес `M` и сохранить его в регистре `R`.

10. Сегмент стека, команды для работы со стеком, команды прерывания.

Стек организовывается с помощью регистров `SS` и `SP`. `SS` хранит адрес начала сегмента стека. `SP` указывает на вершину стека, при добавлении элемента в стек `SP` уменьшается, при удалении — увеличивается.

`push R` — положить значение регистра в стек.

`pop R` — удалить байты с вершины стека, положить в регистр.

`pusha/pora (i386 и новее)` — работать с содержимым сразу нескольких регистров: `AX, BX, CX, DX, SP, BP, SI, DI`.

`pushad/porad (i386 и новее)` — те же регистры, только с префиксом `E`.

К любому значению, хранящемуся в стеке, можно обратиться напрямую:

```
mov BP, SP
mov AX, [BP + 6]      ; слово, первый байт которого находится
                      ; на расстоянии 6 байтов от вершины стека
```

`int` — **команда прерывания**. Ее выполнение приводит к передаче управления `DOS` или `BIOS`, а после завершения какой-то системной обрабатывающей программы управление передается следующей команде.

Действия, происходящие после выполнения `int`, будут зависеть от значения операнда и от значений, хранящихся в регистрах.

Например, прерывания `int 21h` вызовут одну из `DOS`-функций, прерывания `int 10h` — функции видео драйвера, `int 16h` — функции драйвера клавиатуры.

11. Модели памяти, организации программы с помощью точечных директив.

В программе на Ассемблере могут использоваться упрощенные (точечные) директивы.

`.model <модель>` — определяет модель памяти, выделяемой для программы.
`<модель>` — одна из:

- `tiny` — под всю программу один сегмент памяти;
- `small` — по одному сегменту на данные и программу;
- `medium` — под данные один сегмент, под программу несколько;
- `compact` — под программу один сегмент, под данные несколько;
- `large` — под данные и под программу выделяется по несколько сегментов;
- `huge` — позволяет использовать сегментов больше, чем потенциально может поместиться в оперативной памяти.

Пример:

```
1  .model small
2
3  .stack 100h
4
5  .data
6      str1 db 'Line1$'
7
8  .code
9  begin:
10     ; инициализировать DS
11     mov AX, @data
12     mov DS, AX
13
14     ; вывести строку
15     mov AH, 09h
16     mov DX, offset str1
17     int 21h
18
19     ; выйти из программы
20     mov AX, 4C00h
21     int 21h
22 end begin
```

Второй слайд

Вопросы к экзамену по курсу «Машинно-зависимые языки программирования»

- 1 Команды безусловной передачи управления, обращения к подпрограммы и возврата из нее.
- 2 Исполняемые COM-файлы, их отличие от EXE-файлов, примеры.
- 3 Способы адресации операндов, примеры команд.
- 4 Директивы определения данных и памяти.
- 5 Директивы внешних ссылок, организация межмодульных связей.
- 6 Команды двоичной арифметики: сложение, вычитание, умножение и деление.
- 7 Команды побитовой обработки данных: логические операции, операции сдвига.
- 8 Работа с массивами в Ассемблере.
- 9 Команды условной передачи управления, организация циклов в Ассемблере с помощью команд передачи управления.
- 10 Команды для организации циклов в Ассемблере

1. Команды безусловной передачи управления, обращения к подпрограмме и возврата из нее.

Команды передачи управления позволяют изменить ход вычислительного процесса. В языке Ассемблер есть команды условной передачи управления, безусловной передачи управления и команды организации циклов.

Команда безусловной передачи управления:

```
jmp M1      ; прыжок на метку M1, находящуюся в том же  
             ; кодовом сегменте  
  
jmp [BX]     ; прыжок на инструкцию со смещением,  
             ; находящимся в BX
```

Если управление должно передаться на метку в другом кодовом сегменте, то та метка должна быть объявлена `public M2`, а в сегменте, из которого происходит прыжок, должно быть `extrn M2: far`.

Команда `jmp`, если передача управления «дальняя», займет после ассемблирования 5 байтов, а если «ближняя», то 3 байта. Также, если прыжок осуществляется не более, чем на -128 или 127 байтов, то можно использовать «короткую» версию инструкции:

`jmp short <метка>`

Команда обращения к подпрограмме:

`call <имя>`

При обращении к процедуре `near`, в стеке сохраняется адрес команды, следующей за `call` (IP). При обращении к процедуре `far`, в стеке сохраняется адрес текущего кодового сегмента и адрес следующей за `call` команды (CS:IP).

Возврат из процедуры реализуется при помощи команды `ret`. Существуют следующие варианты:

- `ret [N]` — возврат из текущей процедуры;
- `retb [N]` — возврат из процедуры типа `near`;
- `retf [N]` — возврат из процедуры типа `far`.

[N] — необязательный параметр. Определяет количество байт, которое следует удалить из стека, уже после считывания адреса возврата.

2. Исполняемые COM-файлы, отличие от EXE-файлов, примеры.

В DOS существует два типа исполняемых файлов: EXE и COM-файлы.

COM-файл обязательно должен иметь модель памяти `tiny`. Поэтому в нем может быть только один сегмент, объединяющий в себе данные и исполняемые инструкции. Отсюда возникает ограничение на размер COM-файла: 64 КБ.

Кроме этого, COM-файл не содержит в себе никакой дополнительной информации; в то время как у каждого EXE-файла должен быть заголовок, начинающийся со строки `"MZ"` и хранящий информацию такую, как начальные значения CS:IP (точка входа), SS, SP.

Это отражается на процессе написания DOS программ: в начале каждого COM-файла должна быть использована директива `org 100h`, которая «перепрыгнет» специальную структуру PSP размером 256 байт, создающуюся в памяти при запуске любой программы DOS.

Пример COM-программы:

```
1  .model tiny
2
3  .code
4
5  org 100h
6
7      jmp start
8
9      Str1 db 'Hello', '$'
```

```

10 start:
11     mov AH, 09h
12     lea DX, Str1
13     int 21h
14
15     mov AX, 4C00h
16     int 21h
17 end start

```

Аналогичная EXE-программа, модель small:

```

1  .model small
2
3  .stack 100h
4
5  .data
6      Str1 db 'Hello', '$'
7  .code
8  start:
9      mov AX, @data
10     mov DS, AX
11
12     mov AH, 09h
13     lea DX, Str1
14     int 21h
15
16     mov AX, 4C00h
17     int 21h
18 end start

```

Процесс сборки COM-программы будет отличаться от процесса сборки EXE-программы. На этапе связывания, для EXE команда будет:

TLINK.EXE PROG.OBJ

А для COM:

TLINK.EXE /T PROG.OBJ

3. Способы адресации операндов, примеры команд.

- регистровая;

```
mov AX, BX
```

- непосредственная;

```
add AX, 5
```

- прямая;

```
mov AX, ES:0100h  
; ...  
mov CX, DS:String_1
```

- косвенно-регистрая;

```
lea BX, WordArray  
mov word ptr [BX], 0BEEFh  
  
add BX, 2  
mov word ptr [BX], 200
```

- по базе со смещением;

```
lea BX, ByteArray  
mov AX, [BX + 2]  
; ИЛИ  
mov AX, [BX] + 2  
; ИЛИ  
mov AX, 2[BX]
```

- прямая с индексированием;

```
mov AX, WordArray[SI]
```

- по базе с индексированием;

```
mov AX, Arr[SI][DI]
```

- ? косвенная с масштабированием;

```
mov AX, [BX * 4]
```

- ? базово-индексная с масштабированием;


```
mov BX, [SI][DI * 4]
```

- ? базово-индексная с масштабированием и смещением;

```
mov AL, [BX][DI * 2 + 1]
```

Замечание: в квадратных скобках могут упоминаться только регистры BX, BP, SI и DI. Важно помнить, что адреса, хранящиеся в BP, по умолчанию будут связываться Ассемблером с сегментом стека. Поэтому если программист не реализует передачу аргументов в процедуру через стек, то лучше этот регистр не использовать.

4. Директивы определения данных и памяти.

Общий вид директивы определения данных:

```
[<симв. имя>] dX <операнд>[, ...] [; комментарии]
```

где X — это b, w, d, f, q или t.

Операндов может быть один или несколько, каждый операнд должен быть константным выражением (числом), строковым литералом или символом ?. Директива гарантирует выделение указанного объема оперативной памяти под операнд, инициализацию области начальным значением (если не используется ?).

```
.data
R1 db 0, ?, 0 ; выделено 3 байта, нулевой и второй
               ; инициализированы нулями, первый
               ; не инициализируется
```

Другой вариант директивы dX , упрощающий выделение памяти под массивы:

```
[<симв. имя>] dX <конст. выр.> dup(<операнд>) [; комментарии]
```

Она выделит память под <конст. выр.> значений и инициализирует каждую ячейку значением <операнд>.

Замечание: директивой db нельзя определить строковую константу длиной более 255 символов, а вместе с dw нельзя использовать строковые литералы длиной более двух символов.

5. Директивы внешних ссылок, организация межмодульных связей.

Директивы внешних ссылок позволяют организовать связь между различными модулями и файлами, расположенными на диске.

`public <имя>[, ...] [; комментарии]`

Директива `public` определяет указанные имена как глобальные величины, к которым можно обратиться из других модулей. `<имя>` — метка или имя переменной.

Чтобы обратиться к имени из другого модуля, можно использовать директиву `extrn`:

`extrn <имя>:<тип>[, ...]`

Если `<имя>` соответствует переменной, то `<тип>` — это один из `byte`, `word`, `dword`, `fword`, ...

Если `<имя>` — это метка, то `<тип>` — это `near` или `far`.

Пример организации **межмодульной связи**:

```
; Модуль A
public Value

.data
    Value dw 0
; ...

; Модуль B
extrn Value:word
; ...
```

6. Команды двоичной арифметики: сложение, вычитание, умножение и деление.

Сложение и вычитание чисел выполняется по правилам, аналогичным сложению и вычитанию по модулю 2^k , принятым в математике.

В Ассемблере, если в результате получается более k разрядов, то $(k + 1)$ -й пересылается в флаг CF.

$$X + Y = (X + Y) \bmod 2^k = X + Y \text{ и } CF = 0, \text{ если } X + Y < 2^k$$
$$X + Y = (X + Y) \bmod 2^k = X + Y - 2^k \text{ и } CF = 1, \text{ если } X + Y \geq 2^k$$

Пример (для байтов):

$$250 + 10 = 260 = 100000100_2, \text{ и результат: } 00000100_2 = 4, CF = 1$$

$$X - Y = (X - Y) \bmod 2^k = X - Y \text{ и CF} = 0, \text{ если } X \geq Y$$

$$X - Y = (X - Y) \bmod 2^k = X + 2^k - Y \text{ и CF} = 1, \text{ если } X < Y$$

Пример (для байтов):

$$1 - 2 = 1 + 2^8 - 2 = 257 - 2 = 255, \text{ CF} = 1.$$

Важно помнить, что компьютеры не умеют «вычитать». Любая операция вычитания требует предварительного перевода вычитаемого числа в дополнительный код (*two's complement*).

Например, в том же примере $1 - 2$, к результату можно прийти так:

$$\begin{aligned} -2 &= -00000010_2 \\ \text{Доп. код: } 2^8 - 2 &= 11111110_2 \\ 1 - 2 &= 00000001_2 + 11111110_2 = 11111111_2, \text{ CF} = 1, \\ &\text{так как был заем для старшего разряда.} \end{aligned}$$

Результат можно интерпретировать по-разному: как беззнаковое число 255 или как дополнительный код знакового числа -1 . То есть компьютеру не важно, знаковая операция или беззнаковая. Программист сам решает это, исходя из задачи и ориентируясь на флаги.

Инструкции Ассемблера, выполняющие сложение и вычитание:

- add R/M, R/I,
add R, M;
- xadd R/M, R (i486 и новее) — обменять операнды местами, после выполнить сложение;
- adc R/M, R/I,
adc R, M — *add with carry*, сложить два операнда, после прибавить значение флага CF;
- inc R/M;
- sub R/M, R/I,
sub R, M;
- sbb R/M, R/I,
sbb R, M — *subtract with borrow*, вычесть второй операнд из первого, после вычесть значение флага CF;
- dec R/M;

Умножение:

- `mul R/M` — беззнаковое умножение. В зависимости от размера операнда, выполняется одна из операций:
 - если `byte`, то $R/M * AL \rightarrow AX$;
 - если `word`, то $R/M * AX \rightarrow DX:AX$;
 - если `dword`, то $R/M * EAX \rightarrow EDX:EAX$.
- `imul R/M` — знаковое умножение. Есть также версии, появившиеся с i386 (`imul R, R/M/I`) и i186 процессором (`imul R, R/M, I`).

Если в результате умножения $CF = OF = 1$, то результат занимает двойной формат, если $CF = OF = 0$, то результат уместился в размере одного сомножителя.

Деление:

- `div R/M` — беззнаковое деление. В зависимости от размера операнда, значения `AX`, `DX:AX` или `EDX:EAX` делятся на него, и целая часть помещается в `AL`, `AX` или `EAX`, а остаток в `AH`, `DX`, `EDX`.
- `idiv R/M` — знаковое деление.

Программисту нужно следить, чтобы случайно не разделить на 0.

7. Команды побитовой обработки данных: двоичная арифметика.

К командам побитовой обработки данных относятся логические команды, команды сдвига, установки, сброса и инверсии битов.

Как и с арифметическими операциями, оба операнда не могут одновременно находиться в памяти.

Логические операции:

- `and R/M, I`,
 `and R, R/M`;
- `or R/M, I`,
 `or R, R/M`;
- `xor R/M, I`,
 `xor R, R/M`;
- `not R/M` — `not`, в отличие от других логических операций, не меняет значение флагов;

Пример:

```

xor EAX, EAX      ; Еще один способ обнулить значение
                  ; регистра. При ассемблировании
                  ; эта инструкция займет всего
                  ; лишь 2 байта, плюс возможно
                  ; будет работать быстрее, чем
                  ; mov EAX, 0.

```

Общий формат команд арифметического и логического сдвига:

sXY <оп. 1>, <оп. 2> [; комментарий]

где

- X — h (логический), a (арифметический),
- Y — l (влево), r (вправо),
- <оп. 1> — регистр или память,
- <оп. 2> — константа или младшие 5 бит регистра CL.

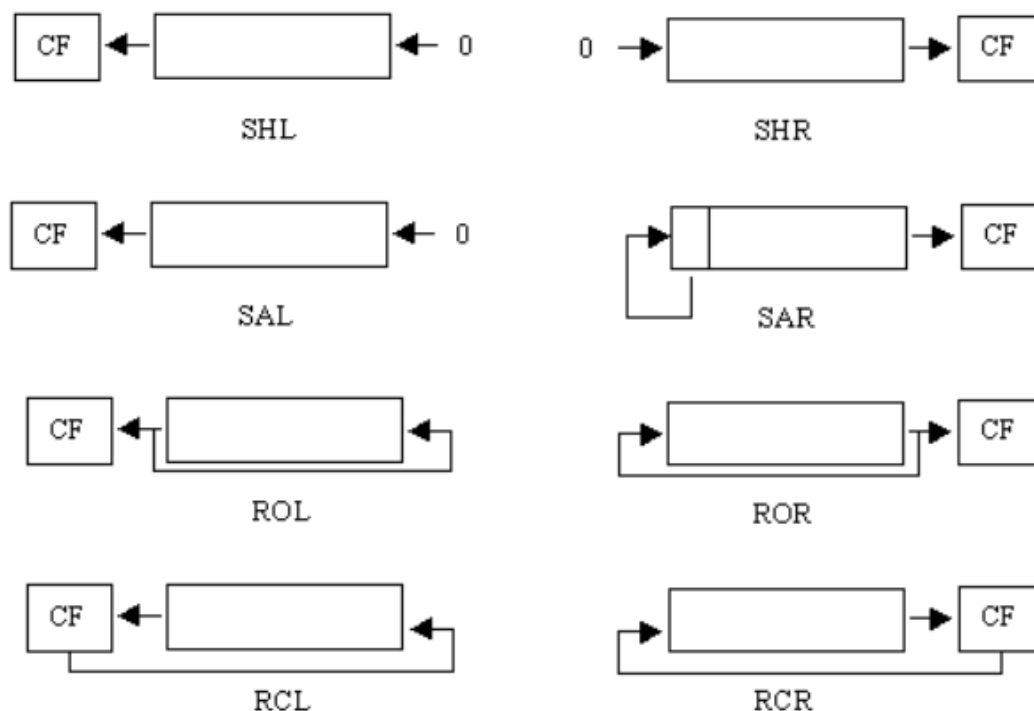


Рис. 2: Различные сдвиги

Логический сдвиг можно также считать *беззнаковым сдвигом*. То есть при сдвиге вправо образовавшиеся пустые места заполняются 0. При арифметическом сдвиге вправо, пустые места заполняются копиями старшего (знакового) бита. Поэтому его называют *знаковым сдвигом*.

При любом типе сдвига последний «ушедший» из числа бит помещается в CF.

Замечание: `shl` и `sal` синонимичны.

Расширенные инструкции сдвигов (i186 и новее):

```
shrd R/M, R, I8/CL
shld R/M, R, I8/CL
```

Содержимое R/M сдвигается на количество разрядов I8/CL, аналогично `shr` и `shl`, но пустые биты заполняются битами из операнда R:

- для сдвига вправо — начиная с самого старшего бита R/M, младшие биты R «задвигаются» в пустые ячейки;
- для сдвига влево — начиная с самого младшего бита R/M, старшие биты R «задвигаются» в пустые ячейки.

(да, это довольно сложно описать. Может быть поэтому А. Г. не пыталась в своей презентации.)

Циклические сдвиги:

- `rol`, `ror` — выполняют циклические сдвиги, причем последний вышедший из разрядной сетки бит оказывается в CF.
- `rcl`, `rcr` — выполняют циклические сдвиги, причем значение флага CF тоже используется и в первую очередь оказывается в числе (см. картинку 2).

8. Работа с массивами в Ассемблере.

Массивы создаются при помощи директив определения данных (чаще всего с использованием `dup(?)`):

```
Arr dw 30 dup(?)
```

Адресация в одномерных массивах:

$$\text{адрес}(\text{Arr}[i]) = \text{Arr} + i * \text{размер}(\text{Arr}[i])$$

Адресация в двумерных массивах (двумерный массив $n \times m$):

$$\text{адрес}(\text{Arr}[i][j]) = \text{Arr} + i * m * \text{размер}(\text{Arr}[i][j]) + j * \text{размер}(\text{Arr}[i][j])$$

размер() — число, соответствующее размеру в байтах одного элемента массива. Например, если массив был задан при помощи dd, то размер каждого элемента будет 4.

Для одномерных массивов удобнее всего будет использовать адресацию *прямую с индексированием*:

Arr[i].

Для двумерных массивов — *по базе с индексированием*:

```
Arr dd 64 dup(128 dup(?))  
; ...  
Arr[BX][DI]          ; Две переменные величины:  
                      ; BX = i * 128 * размер(Arr[i][j])  
                      ; DI = j * размер(Arr[i][j])
```

9. Команды условной передачи управления, организация циклов в Ассемблере с помощью команд передачи управления.

Общий вид **команды условной передачи управления**:

jX <метка> [; комментарии]

где X соответствует одному из условий для передачи управления. Если условие выполняется, то будет совершен прыжок, иначе выполнение продолжится следующей идущей командой.

<метка> должна отстоять от jX не дальше, чем на –128 или 127 байт.

Передача управления осуществляется на основе значений флагов, которые установила предыдущая команда. Например, если после вычитания ZF = 1, то это значит, что результат предыдущей был нулем, и прыжок jz выполнится.

Можно также напрямую «сравнить» два числа командой cmp. Она выполнит обычное вычитание второго операнда из первого, но результат никуда записан не будет. Это установит некоторые флаги в значения, пригодные для сравнения одного числа относительно другого.

Существует также команда test, которая выполнит побитовое and над операндами и установит флаги в нужные значения (более предпочтительна при проверке чисел на равенство).

условие	Для беззнаковых чисел	Для знаковых чисел
>	JA	JG
=	JE	JE
<	JB	JL
> =	JAЕ	JGE
< =	JBE	JLE
< >	JNE	JNE

В принципе, по этой теме это все, что есть в презентации. Если кому-то захочется, можно еще попытаться разобраться в этом.

Для краткости предположим, что непосредственно перед оператором условной передачи управления была выполнена инструкция `cmp OP1, OP2`. Тогда для перехода с условием

- Равно:
 - Для беззнаковых и для знаковых (`je`). `OP1` равен `OP2`, если $ZF = 1$.
- Больше или равно:
 - Для беззнаковых (`jae, jnb`). `OP1` больше или равен `OP2`, если нам не понадобился перенос разряда, то есть $CF = 0$.
 - Для знаковых (`jge, jnl`). Если $SF = 0$ и не было знакового переполнения или если $SF = 1$ и было знаковое переполнение, то `OP1` больше или равен `OP2`. Это условие можно упростить до $SF = 0F$.
- Больше (`ja, jnbe | jg, jnle`): точно такие же условия, что для «больше или равно», но конъюнктивно добавляется условие $ZF = 0$, чтобы исключить равенство `OP1` и `OP2`.
- Меньше:
 - Для беззнаковых (`jb, jnae`). `OP1` меньше `OP2`, если нам понадобился перенос разряда, то есть $CF = 1$.
 - Для знаковых (`j1, jnge`). Если $SF = 0$ и было знаковое переполнение, или если $SF = 1$ и не было знакового переполнения, то `OP1` меньше `OP2`. Или кратко, $SF \neq 0F$.
- Меньше или равно (`jbe, jna | jle, jng`): точно такие же условия, что для «меньше», но дизъюнктивно добавляется условие $ZF = 1$.

Если нужно совершить переход на метку, которая дальше –128 или 127 байтов, то следует инвертировать условие перехода, и сразу после инвертированного условного перехода добавить инструкцию безусловного перехода на нужную метку:

```

; cmp AX, BX
; je M      ; не можем, далеко!

cmp AX, BX
jne Cont
jmp M

Cont:
```



```
; ...
```

Организация цикла с предусловием:

```
l1: ; while x > 0 do S;  
    cmp x, byte ptr 0  
    jle e1  
    ; S  
    jmp l1  
e1:  
    ; ...
```

Организация цикла с постусловием:

```
l1: ; do S while x > 0;  
    ; S  
    cmp x, byte ptr 0  
    jg l1  
e1:  
    ; ...
```

10. Команды для организации циклов в Ассемблере.

Команды для организации циклов:

- `loop <метка>` — $CX = CX - 1$, и если $CX \neq 0$, то перейти на <метка>.
- `loopz <метка>` | `loopnz <метка>` — $CX = CX - 1$. Если $CX = 0$, то выйти из цикла. Иначе проверить флаг ZF:
 - если $ZF = 0$, то выйти из цикла;
 - если $ZF = 1$, то совершить переход на <метка>;
- `loopne <метка>` | `loopnzs <метка>` — $CX = CX - 1$. Если $CX = 0$, то выйти из цикла. Иначе проверить флаг ZF:
 - если $ZF = 0$, то совершить переход на <метка>;
 - если $ZF = 1$, то выйти из цикла.

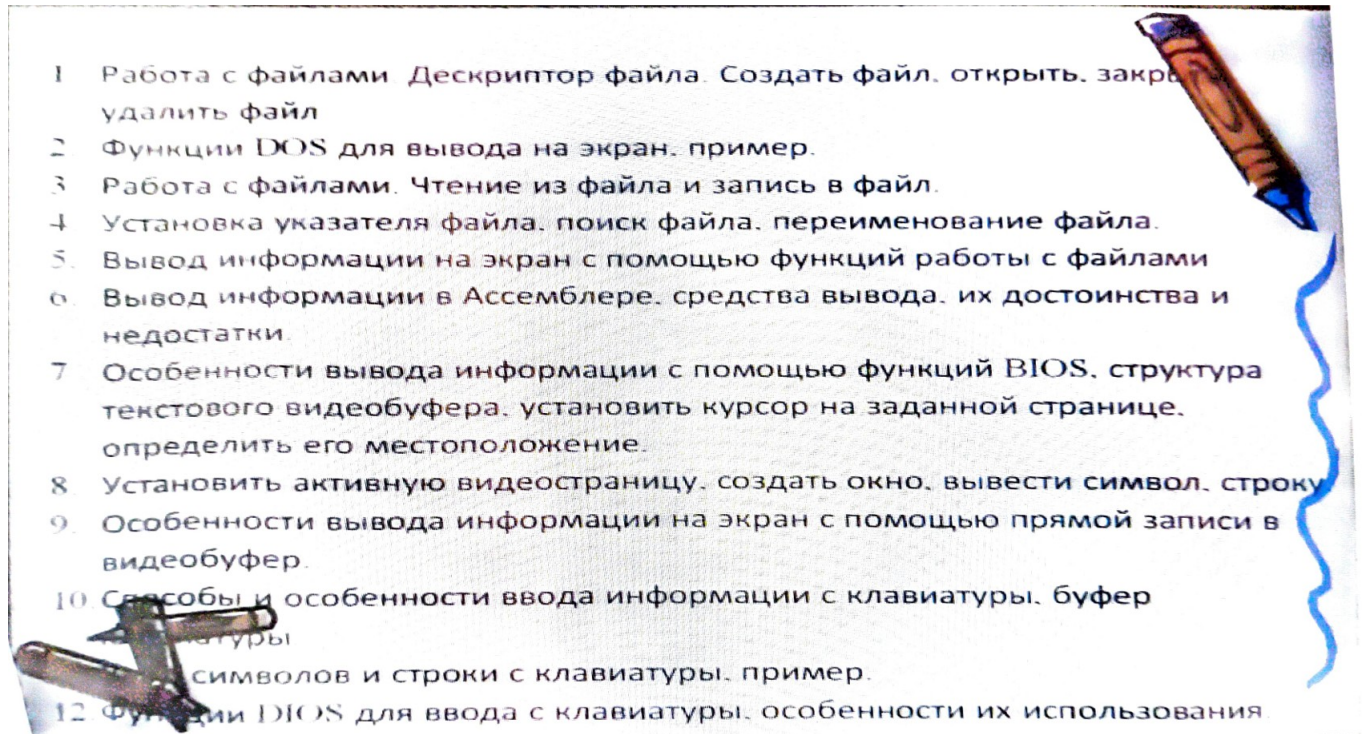
Пример использования:

```
    mov CX, 100
11:    ; ...
    loop 11
12:    ; ...
```

Если CX нужен внутри цикла:

```
    mov CX, 100
11:    push CX
    ; ...
    pop CX
    loop 11
12:    ; ...
```

Третий слайд



1. Работа с файлами. Дескриптор файла. Создать файл, открыть, закрыть, удалить.

Одной из главных функций ОС является организация работы с файлами. Файл на диске — последовательность байтов, пронумерованных с нуля, текущий байт определяется указателем.

Возможен как прямой, так и последовательный доступ к байтам.

Спецификация файла — это строка символов, содержащая букву диска, путь к файлу, имя файла и заканчивающаяся нулем-терминатором.

Открывая файл, ОС создает уникальное 16-разрядное число, называемое номером, идентификатором или **дескриптором файла**, он используется для описания файла и обращения к нему. Первые идентификаторы зарезервированы:

- 0 — STDIN (обычно клавиатура);
- 1 — STDOUT (обычно экран);
- 2 — STDERR (всегда экран);
- 3 — AUX — последовательный порт (обычно COM1);
- 4 — PRN — параллельный порт (обычно LPT1 — принтер).

Стандартный ввод/вывод можно перенаправить на любое устройство или в файл.

Основные **функции** для создания, открытия, закрытия и удаления:

Сори, мои шпоры. Будет sus 😊, если несколько человек придут с одними и теми же распечатками. Просто прикреплю скриншоты ее слайдов.

(Все функции работы с файлами вызываются int 21h.)

Основные функции для работы с файлами

- 1) Создать файл – 3Ch.

При вызове этой функции программист должен определить следующие регистры:

АН ← 3Ch,

DS:DX – адрес ASCIIZ – строки, содержащей С.Ф.

(ASCIIZ – строка – это ASCII – строка, заканчивающаяся нулем).

CX ← атрибуты файла (можно комбинировать)

CX = 0 – без атрибутов, т.е. файл не системный, не директория.

После выполнения функции:

- CF = 0 и AX = дескриптор файла, если функция выполнялась успешно
- CF = 1 и AX = 03h – если путь не найден,
- CF = 1 и AX = 04h – если слишком много открытых файлов,
- CF = 1 и AX = 05h – если доступ к файлу запрещен.
- Если файл с указанной СФ уже существует, то функция 3Ch открывает его все равно, присваивая ему нулевую длину, т.е. информация, содержащаяся в нем, будет затираться новой. Если этого не произошло, если вы не уверены, что такого файла нет на диске, лучше воспользоваться функцией 5Bh.



- 2) Создать и открыть новый файл – 5Bh.

При вызове: АН = 5Bh, CX – атрибут файла,

DS:DX – адрес СФ.

Возврат: CF = 0 и AX – идентификатор файла, открытого для чтения/записи, если не было ошибки;

CF = 1 и AX – код ошибки в противном случае:

AX = 03h, 04h, 05h – также как для 3Ch,

AX = 50h – файл уже существует.

Так что после выполнения функции 5Bh можно сравнить содержимое регистра AX с 50h и принимать решение считывать из этого файла содержимое или заполнять его новой информацией.

- 3) Открыть существующий файл - 3Dh

При вызове: АН = 3Dh,

AL- режим доступа:

0 – открыть для чтения,

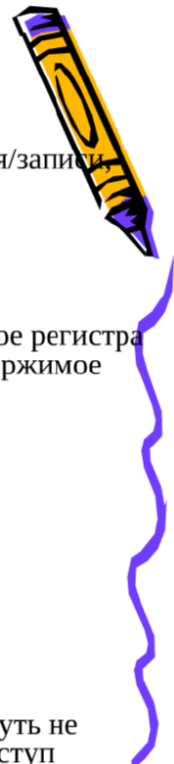
1 – открыть для записи,

2 для чтения и записи.

DS:DX – адрес СФ, CL – атрибуты файла.

Возврат: CF = 0 и AX – дескриптор файла, если не было ошибок и

CF = 1 и AX – код ошибки (02h – файл не найден, 03h – путь не найден, 04h – слишком много открытых файлов, 05h – доступ запрещен) в противном случае.



4)Изменить максимальное число открытых файлов – 67h

При вызове: AH = 67h,

BX - максимальное число открытых файлов (от 20 до 65535)

Возврат: CF = 0 – функция выполнена без ошибки,

CF = 1 и AX - код ошибки: 04h - указанное число меньше количества уже открытых файлов, 08h - DOS не хватает памяти для новой таблицы идентификаторов файлов.

5)Закреть файл – 3Eh.

При вызове: AH = 3Eh, BX = идентификатор файла.

Возврат: CF = 0 - нет ошибки,

- CF = 1 и AX = 6, если указан неверный идентификатор.

6)Удалить файл – 41h.

При вызове: AH = 41h, DS:DX = адрес ASCII строки со СФ.

Возврат: CF = 0, файл удалён,

CF = 1 и AH = 02h – файл не найден, AH = 03h – путь не найден,
AH = 05h – доступ запрещён.



2. Функции DOS для вывода на экран, пример.

Вывод информации на экран в текстовом режиме может быть осуществлен несколькими способами:

1. Используя файловую функцию `int 21h`, AH = 40h.
2. Используя функцию ввода/вывода прерывания `int 21h` —
`int 21h`, AH = 09h,
`int 21h`, AH = 02h.
3. Обратившись к драйверу экрана (`int 10h`).
4. С помощью прямой записи в страницу видеобуфера.

Функции DOS для прерывания INT 21h.

1). Вывод символа на экран в текущую позицию курсора – 02h с проверкой на Ctrl/Break.

Вызов: AH = 02h; DL = ASCII-код выводимого символа

Возврат: AL - код выводимого символа, но если DL = 09h-табуляция, то в AL возвращается 20h.

Если в ходе работы этой функции были нажаты клавиши Ctrl/Break, то вызывается прерывание, которое осуществляет выход из программы.



Вывод текстовой информации на экран

Функция 02h обрабатывает некоторые управляющие символы : вывод символа BEL (07h) приводит к звуковому сигналу; BS (08h) - перемещает курсор влево на одну позицию; HT (09h) - вставляет несколько пробелов; LF (0Ah) - опускает курсор на 1 позицию вниз (перевод строки); CR (0Dh) - переход на начало текущей строки (возврат каретки).

2) Вывод символа на экран в текущую позицию без проверки на Ctrl/Break и без обработки управляющих символов (CR, LF, HT, BS) - 06h

вызов: AH = 06h ; DL = ASCII-код символа

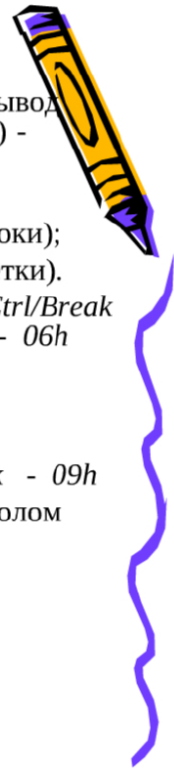
возврат: AL - код выведенного символа, = (DL)

3) Вывод на экран (в стандартное устройство STDOUT) строки символов из буфера пользователя с проверкой на Ctrl/Break - 09h

вызов: AH = 09h; DS:DX = адрес строки, заканчивающейся символом \$(24h)

возврат: AL = 24h - код последнего символа.

- В остальном действие функции 09h аналогично действию функции 02h.



Пример. Вывод на экран таблицы 16×16 всех ASCII-символов.

```
1  .model tiny
2
3  .code
4
5  org 100h
6
7  start:
8      mov CX, 256      ; для цикла
9      mov DL, 0        ; код символа
10     mov AH, 02h      ; номер функции
11 l1:
12     int 21h          ; вызвать функцию
13     inc DL
14     test DL, 0Fh     ; если DL не кратно 16,
15     jnz c1           ; то на c1
16
17     ; иначе
18     push DX          ; сохранить код в стеке
19
20     ; Вывод CRLF
21     mov DL, 0Dh
22     int 21h
23     mov DL, 0Ah
24     int 21h
25
26     pop DX           ; восстановить код из стека
```



```
27 c1:
28     loop 11
29 end start
```

3. Работа с файлами. Чтение из файла, запись в файл.

7) Чтение из файла или устройства – 3Fh

При вызове: AH = 3Fh; BX = идентификатору файла; CX = количеству байт, которые необходимо переслать, DS:DX - адрес памяти для приёма данных.

Возврат: CF = 0 и AX = количеству реально переданных байтов, если ошибки нет, или в противном случае:

- CF = 1 и AX = 05h – если доступ запрещён, AX = 06h – неправильный идентификатор (дескриптор в BX).

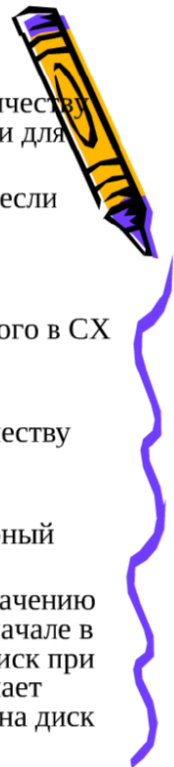
Количество считанных байтов может оказаться меньше указанного в CX при достижении конца файла.

8) Запись в файл или устройство – 40h.

При вызове: AH = 40h, BX = идентификатору файла, CX = количеству байт, DS:DX = адресу буфера с передаваемыми данными.

Возврат: CF = 0 и AX - количество записанных байт, или CF = 1 и AH = 05h – доступ запрещён, AH = 06 - неверный идентификатор.

Если при записи в файл CX = 0, файл обрывается по текущему значению указателя. При записи в файл информация записывается вначале в специальный буфер OS, из которого она сбрасывается на диск при завершении файла или, если количество информации превышает размер сектора диска. Для немедленного сброса из буфера на диск можно использовать функцию 68h.



4. Установка указателя файла, поиск файла, переименование файла.

Установка указателя файла:

Возврат: CF = 0 и CX:DX = новое значение указателя в байтах от начала файла, если нет ошибки. CF = 1 и AX = 06h, если неправильный дескриптор.

Эту функцию можно использовать для определения длины файла, вызвав ее со следующими параметрами:

CX = DX = 0, AL = 2 и в результате выполнения функции в CX:DX будет реальная длина файла в байтах. Указатель можно установить за реальными пределами файла: если задать отрицательный номер байта, то следующая операция выдаст сообщение об ошибке, а если в результате выполнения функции значение указателя окажется больше длины файла, то следующая операция записи увеличит размер файла.

Поиск и переименование файла:

Поиск файлов, имеющих короткие и длинные имена выполняются с помощью различных функций DOS. Рассмотрим функции для коротких имен файлов.

12) Найти первый файл – 4Eh.

При вызове: AH = 4Eh,

CX = атрибуты файла (могут комбинироваться),

DS:DX = адрес ASCIIZ – строки со СФ, которая может содержать обобщающие символы * и ?.

Возврат: CF = 0 и имя, расширение и информация о найденном файле помещаются в область ДТА (Disk Transfer Area), если файл найден,

или CF = 1 и AX = 02h, если файл не найден, AX = 03h, если путь не найден, AX = 12h, если неправильный режим доступа.

Область памяти ДТА - это 128-байтный буфер, который находится в PSP со смещением от начала этого блока по умолчанию равным 0080h, но его можно и переопределить с помощью функции 1Ah.

Блок PSP располагается перед COM и EXE - файлами, адрес этого блока, размером в 256 байт при запуске EXE-, COM - файлов содержится в регистрах DS и ES и CS.



13). Найти следующий файл – 4Fh.

При вызове: AH = 4Fh,

ДТА = содержит данные от предыдущего вызова функции 4Eh или 4Fh.

Возврат: CF = 0 и ДТА содержит данные о следующем найденном файле, если ошибки нет, и CF = 1 и AX = коду ошибки, если была ошибка.

4Eh - находит первый файл, соответствующий указанному в DS:DX шаблону, а 4Fh – находит следующий, соответствующий этому шаблону. Чтобы найти все такие файлы, функция 4Fh выполняется до тех пор, пока CF станет равным 1. (CF=1).

14) Переименование файла-56h.

При вызове: AH = 56h, DS:DX = адресу текущей спецификации, записанной в формате ASCIIZ-строки,

ES:DI = адрес новой спецификации, также в формате ASCIIZ строки.



5. Вывод информации на экран с помощью функций работы с файлами.

Вывод на экран информации с помощью функции int 21h, AH = 40h:

```
1  .model tiny
2
3  .code
4
5  org 100h
6
7  start:
8      mov AH, 40h      ; функция записи в файл
9      mov BX, 2        ; STDERR
10     mov DX, offset Msg ; буфер
11     mov CX, Len_Msg   ; размер буфера
12     int 21h
13     ret
14
15     ; символ $ можно использовать как обычный символ
16     Msg db "$ Here comes the money! $"
17     Len_Msg = $ - Msg ; длина строки
18 end start
```

Здесь напрямую указан дескриптор вывода (2), поэтому перенаправить вывод этой программы куда-либо не получится.

6. Вывод информации в Ассемблере, средства вывода, их достоинства и недостатки.

Несколько способов вывода информации:

1. **Файловый вывод** (функции DOS работы с файлами `int 21h`). Достоинства: диск — большее хранилище по сравнению с оперативной памятью. Недостатки: медленная скорость работы.
2. **Графический вывод** достигается следующими путями:
 - Функциями DOS вывода текста на экран (`int 21h`). Достоинства: просты в использовании. Недостатки: не самая быстрая скорость работы, нет возможности изменить цвет выводимой информации.
 - Функциями драйвера экрана (`int 10h`). Достоинства: появляется возможность менять атрибуты выводимых символов, выводить информацию в любом месте экрана. Недостатки: медленная скорость работы, большое количество обращений к DOS функциям.
 - Прямым изменением видеопамати (в текстовом режиме). Достоинства: максимально возможная быстрота обновления изображения, возможность изменять атрибуты символов. Недостатки: от программиста требуется понимание того, как устроена видеопамать, и знание некоторых «волшебных адресов».

uhh, actually 😬 Если очень постараться, можно выводить информацию еще при помощи клавиатуры: есть возможность изменять состояние лампочек-индикаторов Num Lock, Caps Lock и Scroll Lock.

7. Особенности вывода информации с помощью функций BIOS, структура текстового видеобуфера, установить курсор на заданной странице, определить его местоположение.

Вывод информации на экран при помощи функций BIOS позволяет пользователю выводить текст в любом месте экрана и изменять атрибуты символов.

Информация, отображаемая на экране, хранится в специальной области оперативной памяти, называемой **видеобуфером**. По умолчанию, в ней одновременно хранятся восемь независимых изображений экрана, называемых **страницами**, с номерами от 0 до 7. Только одна из этих страниц может быть активна в данный момент времени, изменение страницы сразу отражается на изображении.

Существует несколько **видеорежимов**, которые бывают текстовыми и графическими. Режимы различаются разрешением и глубиной цвета.

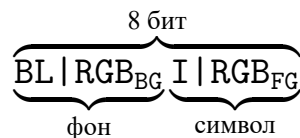
Дисплей						
Режим	Строки×Столбцы	Тип	Адрес	Страницы	Разрешение	Цвета
00	25×40	Цветной	B800	0–7	360×400	16
01	25×40	Цветной	B800	0–7	360×400	16
02	25×80	Цветной	B800	0–3	720×400	16
03	25×80	Цветной	B800	0–3	720×400	16
07	25×80	Монохромный	B000	0	720×400	

Текстовые страницы адаптера EGA хранятся по адресам

- 0: B8000h–B8F40h,
- 1: B9000h–B9F40h,
- 2: BA000h–BAF40h,
- ...,
- 7: BF000h–BF400h.

По умолчанию используется текстовый режим 3 со страницей 0. То есть у программиста в распоряжении 25 строк по 80 символов, 16 цветов.

Структура атрибута символа:



- 7-й бит BL — (*blink*) мерцание фона.
 - 0: не мерцает;
 - 1: мерцает.
- 4–6 биты RGB_{BG} — цвет фона.
- 3-й бит I — (*intensity*) яркость цвета символа.
 - 0: темнее;
 - 1: светлее.
- 0–2 биты RGB_{FG} — цвет символа.

Каждая **текстовая страница** представляет собой последовательность из 16-битных значений. Младший байт значения — ASCII-код символа, старший байт — атрибут символа.

B8000h÷0001	B8000h÷0002	B8000h÷0003	B8000h÷0004
ASCII-код	Атрибут	ASCII-код	Атрибут
← символ 0	→ ←	символ 1	→

Функции int 10h для работы с курсором:

Установить курсор в заданную позицию на указанной странице – 02h

вызов: АН = 02h; ВН = номер страницы;

ДН = номер строки; DL = номер столбца

3) Определить положение курсора и его размер на указанной странице – 03h

вызов: АН = 03h; ВН = номер страницы

возврат: ДН и DL – номера строки и столбца текущей позиции курсора

СН и CL – 1 и последняя строки, занимаемые курсором (его размер).

8. Установить активную видеостраницу, создать окно, вывести символ, строку.

4) Задать активную видеостраницу – 05h

вызов: АН = 05h; AL = номер страницы.

5) Инициализация окна и его прокрутка вверх – 06h.

Выводит окно, заданного размера, заполняя его пробелами с заданным атрибутом или прокручивает содержимое окна вверх на заданное число строк на активной видеостранице.

Вызов: АН = 06h

AL = количеству строк прокрутки, если AL= 0, окно очищается, ВН = атрибут, СН и CL = координаты у и х

 ВН = координаты y и x верхнего угла DH и DL= координаты y и x нижнего правого угла окна

6) Инициализация и прокрутка окна вниз-07h.

Вызов: АН = 07h остальные параметры аналогичны функции 06h

8) Вывести символ с текущим атрибутом-OAh.

Вызов: AH = OAh; BH = номер страницы;

AL = ASCII-код выводимого символа, CH- количество повторений символа.

Используется атрибут символа, который был в этой позиции ранее.

10) Вывести строку символов с указанной позиции на текущую страницу
13h.

При выводе управляющих символов выполняются соотв - ие им действия.

Вызов: AH = 13h; AL = режим:

Бит 1 – после вывода переместить курсор в конец строки

0 – курсор не смещается

Строка состоит только из ASCII – символов,

атрибут находится в BL.

Биты 2 и 3 – строка состоит из чередующихся кодов символов и атрибутов; 2

– курсор не смещается после записи, 3 – смещается.

BH- номер страницы. ES:BP – адрес строки в памяти.

CX – длина строки без учета байтов-атрибутов

DH и DL – номер строки и столбца соответственно.



9. Особенности вывода информации на экран с помощью прямой записи в видеобуфер.

Изображение, отображаемое на экране монитора, формируется непосредственно из данных, хранящихся в активной странице графического режима. Следовательно, программист может не использовать никаких DOS или BIOS функций, чтобы изменить изображение. Достаточно лишь знать начальный адрес страницы, из которой будут браться данные.

Можно положить этот адрес в какой-то из свободных сегментных регистров, например ES:

```
; загрузить адрес начала страницы в ES
mov AX, 0B800h
mov ES, AX

; mov ES:0001h, byte ptr 00001001b
; ...
```

Задача вывода всех ASCII-символов, решенная при помощи прямого доступа к видеопамяти:

```

1  .model tiny
2  .386
3
4  .code
5
6  org 100h
7
8  start:
9      mov AX, 0003h    ; видеорежим 3, очистка
10                     ; экрана, курсор в (0, 0)
11      int 10h
12
13      cld             ; DF = 0
14      mov EAX, 1F201F00h ; символ '\0' с атрибутом 1Fh
15                     ; и пробел ' ' с атрибутом 1Fh
16      mov BX, 0F20h    ; пробел ' ' с атрибутом 0Fh
17      mov CX, 255      ; счетчик цикла
18      mov DI, offset Table ; адрес таблицы символов в ES:DI
19  l1:
20      stosd
21      inc AL
22      test CX, 0Fh
23      jnz c1
24
25      push CX
26
27      ; Заполнить остаток строки пробелами
28      mov CX, 80 - 32
29      xchg AX, BX
30      rep stosw        ; записать пробел
31      xchg BX, AX
32      pop CX
33  c1:
34      loop l1
35
36      stosd            ; записать последний символ и пробел
37
38      ; Вывод содержимого таблицы
39      mov AX, 0B800h    ; адрес начала видеопамяти
40      mov ES, AX
41      xor DI, DI        ; теперь ES:DI указывает на начало
42      mov SI, offset Table
43      mov CX, 15 * 80 + 32 ; размер Table
44      rep movsw         ; запись таблицы в видеопамять
45      ret
46  Table:
47  end start

```

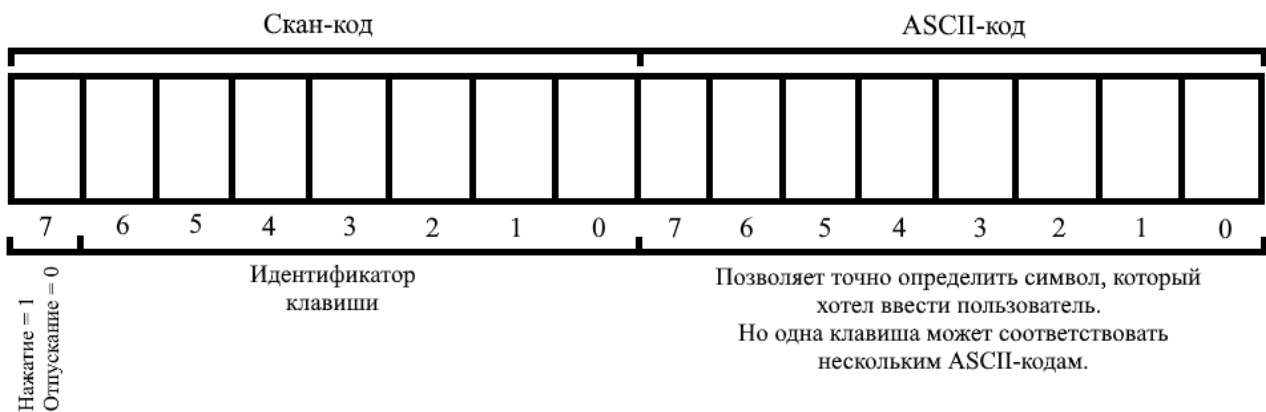
10. Способы и особенности ввода информации с клавиатуры, буфер клавиатуры.

Способы ввода информации с клавиатуры:

1. Функция DOS `int 21h`, `AH = 3Fh`.
2. Другие специальные функции DOS прерывания `int 21h`.
3. Использовать функции драйвера клавиатуры `int 16h`.

Нажатие или **отпускание** клавиши (или сочетания клавиш) приводит к генерации 8-разрядного скан-кода. Этот скан-код объединяется с другим 8-разрядным значением — ASCII-кодом клавиши — после чего это 16-разрядное число записывается в буфер клавиатуры.

Младший байт — ASCII-код, старший байт — скан-код:



При обработке ввода важно учитывать несколько факторов:

- Если пользователь нажал клавишу или сочетание клавиш, за которым не закреплено никакого ASCII-символа, то в буфер клавиатуры запишется **расширенный ASCII-код** символа: в 16-разрядном коде, представленном выше, младший байт будет нулевым.
- Клавиатуры бывают разные, и поэтому функции для обработки ввода `int 16h` могут отличаться для 84, 102 и 122-клавишных клавиатур.

11. Ввод символов и строки с клавиатуры, пример.

Пример использования функции `int 21h`, `AH = 0Ah`, которая вводит строку символов с ожиданием, эхом и проверкой на `Ctrl/Break`:

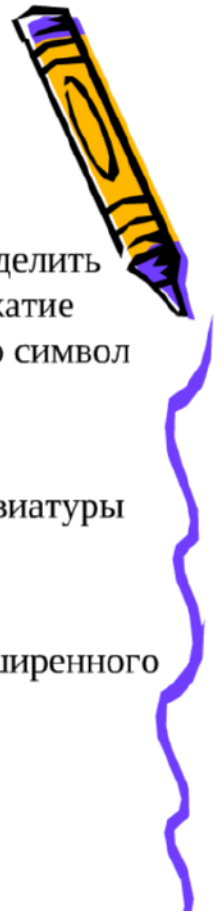
```

1  .model small
2  .386
3
4  .stack 100h
5
6  ; Наша DOS-функция ждет от пользователя ввода, пока он
7  ; не нажмет клавишу Enter (она же клавиша Return).
8  ; То есть помимо текста,
9  ; который ввел пользователь, в буфере окажется еще
10 ; символ возврата каретки -- 0Dh. Отсюда и +1 к размеру
11 ; буфера.
12 Input_Buffer struc
13     Buffer_Max_Len db 64 + 1      ; максимальная длина ввода
14     Buffer_Len db 0               ; истинная длина ввода
15     Buffer_Data db 64 + 1 dup(0) ; буфер для ввода
16 Input_Buffer ends
17
18 .data
19     IBuf Input_Buffer <>
20     IBuf_Address dd IBuf
21
22 .code
23 start:
24     ; Загрузить "дальний указатель", т. е. установить
25     ; DS = адрес начала сегмента, содержащего IBuf_Address,
26     ; DX = смещение IBuf_Address внутри сегмента.
27     ; (https://c9x.me/x86/html/file\_module\_x86\_id\_152.html)
28
29     ; DOS-функция, которую мы собираемся использовать,
30     ; принимает не *смещение буфера* в сегменте данных,
31     ; а *указатель на начало буфера*.
32     lds DX, IBuf_Address
33     mov AH, 0Ah
34     int 21h
35
36     ; ...
37 end start

```

12. Функции BIOS для ввода с клавиатуры, особенности их использования.

Функции BIOS для ввода с клавиатуры.



Функции BIOS предоставляют большие возможности для работы с клавиатурой, чем функции DOS, например, DOS не может определить момент отпускания нажатой клавиши, не может определить нажатие комбинации управляющих клавиш, не может поместить в буфер символ так, как если бы его ввел пользователь.

Функции BIOS вызываются прерыванием INT 16h.

1). Чтение 16-разрядного кода из клавиатуры в зависимости от клавиатуры (84/102/122) : 0 или 10h или 20h

Вызов: AH = 0 или 10h или 20h

Возврат: AH = скан-код или информационный (старший) байт расширенного ASCII-кода

AL = ASCII-код символа или младший (нулевой) байт расширенного кода.

Если буфер пуст, ожидает ввода.



Функции BIOS для ввода с клавиатуры.

2). Проверка наличия символа и чтение 16-разрядного кода без извлечения его из буфера клавиатуры: 01h (11h, 21h).

Вызов: AH = 01h (11h, 21h)

Возврат: ZF = 0 и AH = скан-код, AL = ASCII-код символа,
или ZF = 1, если символа нет.

3). Считывание состояния клавиатуры : 02h (12h, 22h)

Вызов: AH = 02h (12h, 22h)

Возврат: AL = байт состояния клавиатуры для 02h

AH = старший байт состояния клавиатуры для 12h и 22h

AL = разряд 0 – нажата правая клавиша Shift

1 – нажата левая клавиша Shift

2 - ----- любая Ctrl

3 - -----Alt

4 - включена ScrollLock

5 – включена NumLock

6 - ----- CapsLock

7 - ----- Ins



Функции BIOS для ввода с клавиатуры.

Содержимое этих байтов AH и AL постоянно хранятся в памяти.

Вместо вызова прерывания иногда удобнее считывать значения их напрямую из памяти и, если записать новые значения, то BIOS изменит состояние клавиатуры. Напрямую можно обращаться к любому байту буфера клавиатуры, значения которых хранятся в специальном участке памяти. Но прямой доступ не намного быстрее работы функций BIOS.

4). Поместить символ в буфер клавиатуры – 05h.

Вызов: AH = 05h, CH = скан-код символа, CL = ASCII- код символа

Возврат: AL = 00 – операция выполнена успешно

AL = 01h – если буфер переполнен

Пример1. Ввод строки с помощью функции 0 (10h, 20h)

Программа вводит 15 символов и записывает их в память по адресу Str.



1. Структуры в Ассемблере и их использование.
2. Записи в Ассемблере, их описание и использование.
3. Работа с подпрограммами в Ассемблере, способы передачи параметров.
4. Передача параметров в подпрограммы по ссылке и по значению.
5. Передача параметров через стек, пример.
6. Локальные параметры в процедуре.
7. Организация работы со строками, установка и сброс флагов.
8. Команды для работы со строками – MOVSB, LODSB, STOSB.
9. Команды работы со строками: CMPSB, SCASB, INSB, OUTSB.
10. Организация работы со строками переменной длины в Ассемблере.
11. Блоки повторений и макрооператоры в Ассемблере.
12. Директивы условной генерации и их назначение.
13. Макросы в Ассемблере, их описание и использование.
14. Примеры использования директив условной генерации.
15. Логические выражения в директивах условной генерации.

1. Структуры в Ассемблере и их использование.

Структура состоит из полей различного типа и длины, занимая последовательные байты памяти.

После объявления структуры можно создавать переменные, для которых структура является прототипом.

Описание структуры:

```
<имя типа> struc
    <описание поля>
    ...
<имя типа> ends
```

<имя типа> — имя, которое будет использоваться при описании переменных типа этой структуры.

Для описания полей используются директивы `dX`. Но имена, используемые с этими директивами, не локализованы, и поэтому должны быть уникальны для всей программы. Поля не могут иметь тип структуры.

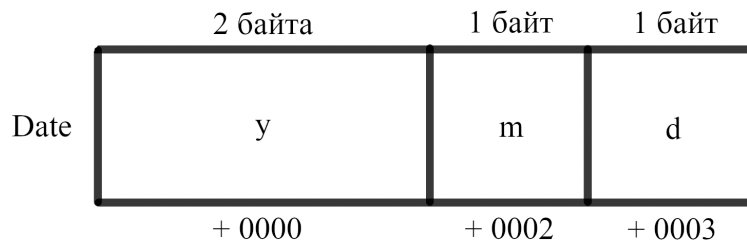
Поля могут иметь значения по умолчанию, а могут не иметь (?).

Пример объявления структуры:

```

Date struc
    y dw 2000
    m db ?
    d db 28
Date ends

```



Использование структуры:

имя_переменной имя_структуры <начальное_значение, ...>

< > — служебные символы, внутри которых через запятую указываются **начальные значения полей** (? , строковые литералы, константные выражения или ничего).

Пример объявления переменных структурного типа:

```

dt1 Date <?, 6, 4>
dt2 Date <1999, , >
dt3 Date <, , >

```

В приоритете всегда будет начальное значение, указанное при объявлении переменной, даже если в объявлении структуры у него есть значение по умолчанию.

При объявлении переменной, можно не указывать начальные значения *последних* полей и сразу поставить знак >. Но для *первых* полей нужно проставить запятые, означающие «пустоту»:

```

dt4 Date <1980> ; то же самое, что <1980, , >
dt5 Date <, , 5> ; нельзя просто <5>

```

Можно не указывать никаких начальных значений, а просто использовать значения по умолчанию:

```

dt6 Date < >

```

Пример объявления массива переменных структурного типа:

```
; Массив из 25 элементов типа Date.  
; Первый элемент имеет начальное значение < , 4, 1>.  
; Остальные 24 элемента используют значения по умолчанию.  
dts Date < , 4, 1>, 24 dup(< >)
```

С начальными значениями есть сложности:

1. Нельзя присваивать начальное значение полям-массивам (`arr dw 1, 2, 3`). Но в случае `db` можно использовать в качестве начального значения строку.
2. Поле-строка, объявленное любым образом (`s db 10 dup(?)`, `t db 21 dup(0)`, `v db 's', 't', 'r'`), может иметь начальное значение, но
 - Начальное значение не должно быть длинее значения по умолчанию.
 - Если начальное значение короче значения по умолчанию, то оно в конце дополняется пробелами.

Обращение к полям структуры:

имя_переменной.имя_поля

Оператор `.` вычисляет адрес так:

<адресное выражение> + <смещение поля в структуре>

<адресное выражение> может быть любой сложности:

```
mov AX, (dts + 8).y  
; ...  
mov SI, 8  
inc (dts[SI]).m  
; ...  
lea BX, dt1  
mov [BX].d, 10
```

Полезный оператор `type` разрешается в размер переменной:

```
type Date = type dt1 = 4  
type dt1.m = type m = 1  
type (dts[SI]).m = type (dts[SI].m) = 1  
type dts[SI].m = type dts = 4
```

Пример обращения к полям структуры:

```

1  .model tiny
2
3  .code
4
5  org 100h
6
7  start:
8      ; вывод "hello\r\n"
9      mov AH, 9
10     mov DX, offset Message
11     int 21h
12
13     lea BX, st1          ; адрес первой записи
14     mov CX, 2            ; количество повторений внешнего цикла
15 12:
16     push CX
17     mov SI, 0
18     mov CX, 3            ; количество повторений внутреннего цикла
19 11:
20     ; вывод одного поля структуры
21     push CX
22     lea DX, [BX][SI]     ; адресация по базе с индексированием
23     int 21h
24
25     ; поскольку первое и второе поле имеют
26     ; длину 9 байт, то прибавить 9 к SI
27     add SI, 9            ; переход к следующему полю
28     pop CX
29     loop 11
30
31     add BX, type TStudent ; переход к следующей переменной
32     pop CX
33     loop 12
34
35     ret
36
37     Message db "hello", 0Dh, 0Ah, "$"
38
39     TStudent struc        ; описание структуры TStudent
40         s db 9 dup(?)
41         f db 9 dup(?)
42         i db 9 dup(?)
43     TStudent ends
44
45     ; две переменные структурного типа TStudent
46     st1 TStudent <"student $", "Ivanov $", "Ivan, $">
47     st2 TStudent <"student $", "Petrov $", "Petr $">
48 end start

```


2. Записи в Ассемблере, их описание и использование.

Запись — это упакованные данные, которые занимают не отдельные, полные ячейки памяти, а части ячеек.

Запись в Ассемблере может в общей сложности занимать байт или слово, поля записи — группы последовательных битов. Если запись получается меньше байта или слова, поля записи расположатся в младших битах ячейки.

Также, как и со структурами, записи сначала описывают, а затем объявляют переменные типа записи.

Описание записи:

```
<имя типа записи> record <имя поля>:<размер>[=<значение>] [, ...]
```

- <значение> — константное выражение, значение по умолчанию для поля. Не может быть ?. Если не указано, то полю присваивается 0.
- <размер> — константное выражение, размер поля в битах.
- <имя поля> — символическое имя, уникальное в рамках программы.

Создание переменных типа записи:

```
имя_записи имя_типа_записи <начальное значение, ...>
```

Начальными значениями могут быть константное выражение, ? или ничего.

Начальное значение ? приравнивается к 0. «Пустота» означает, что нужно использовать значение по умолчанию.

Также, как и для структур, можно опускать *последние* начальные значения.

Допускается использование директивы dup.

Использование:

```
Date record y:7, m:4, d:5
; ...
dt1 Date <80, 7, 4>
dt2 Date <0>

; ...
mov AX, dt1
mov dt2, AX
```

Для работы с отдельными полями используются операторы width и mask:

```
width <поле>
width <запись>
```

width возвращает размер поля или всей записи в битах, в зависимости от операнда.

mask <поле>
mask <запись>

mask в зависимости от размера записи, возвращает байт или слово, соответствующие маске, которая содержит 1 в тех разрядах, которые принадлежат полю или записи.

Пример:

```
Date record y:7, m:4, d:5
; ...
dt Date < >
; ...

; Найти тех, кто родился 1 числа месяца.
; dt хранит дату, с которой работаем.

; ...
11:
mov AX, dt
and AX, mask d
cmp AX, 1
je yes
no:
; ...
yes:
; ...
```

Именам полей Ассемблер присваивает количество бит, на которое нужно совершить сдвиг вправо, чтобы значение поля оказалось в младших битах ячейки.

3. Работа с подпрограммами в Ассемблере, способы передачи параметров.

В Ассемблере один вид подпрограмм — процедуры.

Общий вид **процедуры**:

```
<имя процедуры> proc <параметр>
    <тело процедуры>
ret
<имя процедуры> endp
```

где

- <параметр> — near или far.

Размещать подпрограммы лучше так, чтобы исполнение их кода не происходило без команды call. То есть до первой исполняемой команды или после ret главной подпрограммы.

Параметры можно передавать разными способами:

- по значению;
- по ссылке;
- по возвращаемому значению;
- по результату;
- отложенными вычислениями.

Также различают передачу

1. **через регистры** — параметры записываются в регистры, процедура сразу их использует. Самый простой и эффективный способ. Но нельзя передать большое число параметров, так как свободные регистры могут закончиться
2. **в глобальных переменных** — в сегменте данных объявляется глобальная переменная, позже процедура к ней обращается. Недостаток этого способа в том, что становится невозможна рекурсия.
3. **через стек** — параметры отправляются в стек командами `push`, а подпрограмма извлекает их из стека с помощью регистров `SP` и `BP`. Оптимальный способ, используется современными языками программирования.
4. **в потоке кода** — данные размещаются в вызывающем коде сразу за командой `call`. Подпрограмма, используя адрес возврата, который лежит в стеке (бывшее значение `IP` или `CS:IP`), извлекает их. Этот способ менее эффективен, чем уже перечисленные.
5. **в блоке параметров** — в сегменте данных создается специальный блок, содержащий все необходимые параметры. Вызываемая подпрограмма получает адрес этого блока любым вышеперечисленным способом или даже из другого блока параметров.

Пример такой передачи данных: функция `DOS`, находящая первый файл, удовлетворяющий ASCIIZ-спецификации (`int 21h`, `AH = 4Eh`). Эта функция возвращает блок `DTA`, если файл был найден.

<https://stanislavs.org/helppc/dta.html>

4. Передача параметров в подпрограммы по ссылке и по значению.

При передаче параметров **по значению** процедура работает с копией данных, и входные параметры, хранящиеся в памяти, не изменяются.

```
; Процедура, определяющее наибольшее из двух чисел.  
; Одно число передается через регистр AX, другое -- через BX.  
; Результат возвращается через регистр AX.
```

```
Max proc
```

```
    cmp AX, BX  
    jge e1  
    mov AX, BX
```

```
e1:
```

```
    ret
```

```
Max endp
```

```
; Вызов:
```

```
mov AX, a
```

```
mov BX, b
```

```
call Max
```

При передаче параметров **по ссылке**, процедура получает адрес области памяти, где хранится значение, поэтому в него можно внести изменения.

```
; Процедура, выполняющая операцию  $x = x / 16$ ,  
; где x хранится в области памяти по  
; адресу [BX]. (x -- машинное слово)
```

```
Proc_div16 proc
```

```
    push CX
```

```
    mov CL, 4
```

```
    shr word ptr [BX], CL    ;  $x = x / 16$ 
```

```
    pop CX
```

```
    ret
```

```
Proc_div16 endp
```

```
; Вызов:
```

```
lea BX, a
```

```
call Proc_div16
```

Массивы, располагающиеся в блоке данных, удобно передавать процедуре по ссылке на их начало.

Ниже пример подпрограммы, находящей максимальный элемент в массиве беззнаковых байтов. Она принимает адрес начала массива через регистр BX, количество элементов массива через регистр CX, а результат возвращает в регистре AL.

```
Max proc
```

```
    push BX    ; Сохраним значения в стеке на случай, если
```

```
    push CX    ; вызывающей программе не нужно,
```

```
    ; чтобы мы их изменяли
```

```
    xor AL, AL    ; текущее максимальное значение = 0
```

```
l1:
```

```
    cmp [BX], AL
```

```
    jna c1    ; если элемент не больше AL, то продолжать
```

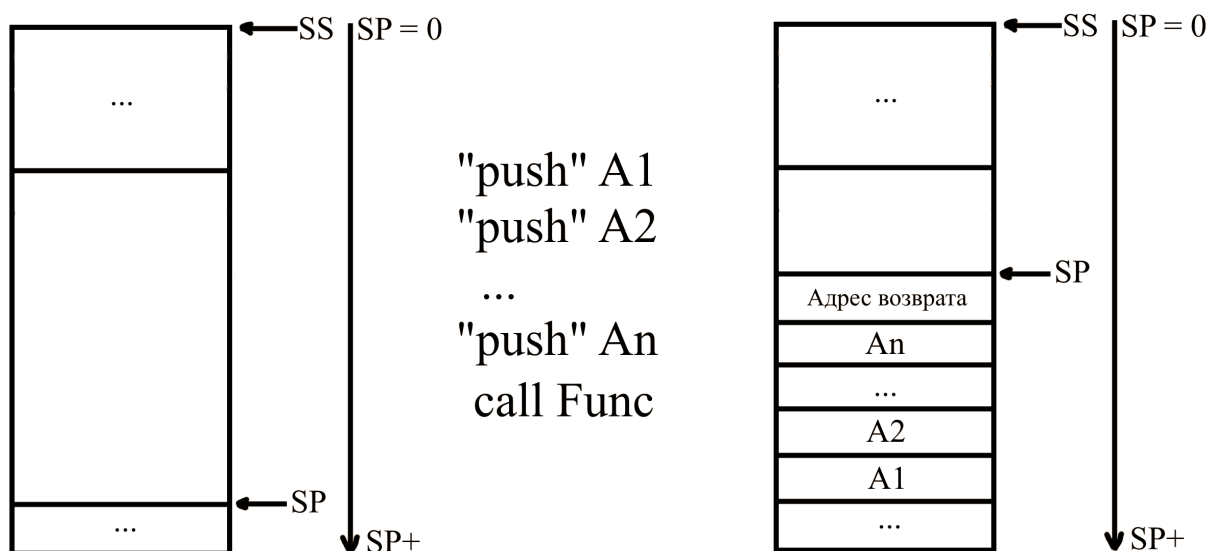
```

        mov AL, [BX]
c1:      inc BX          ; перейти к следующему элементу
        loop l1

        pop CX
        pop BX
        ret
Max endp

```

5. Передача параметров через стек, пример.



Адрес возврата — адрес следующей за `call` команды. На рисунке отображено состояние стека до и после вызова процедуры, принимающей n параметров через стек.

Важно: Стоит помнить, что команда `push` принимает в качестве единственного операнда только регистры, там не может использоваться имя переменной. Кроме того, с помощью команды `push` нельзя положить в стек значение меньше машинного слова.

Обращение к таким переменным в процедуре:

```

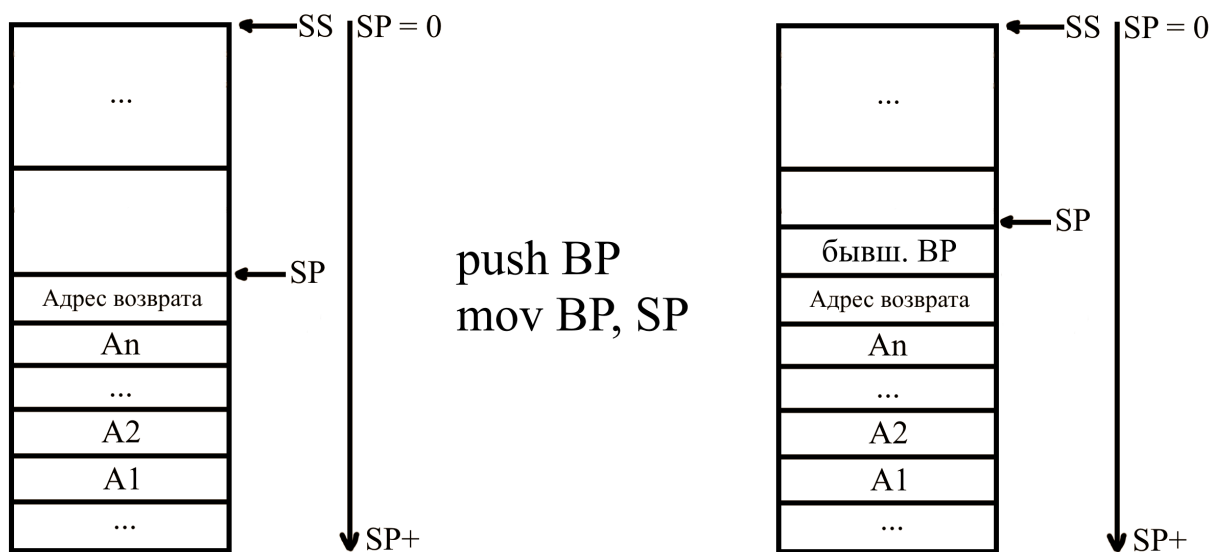
Func proc near
    push BP          ; сохраним старое значение BP в стеке
    mov BP, SP
    ; [BP + 4] = n-й параметр
    ; [BP + 6] = (n - 1)-й параметр
    ; ...

```

```

    pop BP
    ret 2 * n      ; очистка стека от n машинных слов
Func endp

```



Пример подпрограммы, обнуляющей переданный ей массив байтов. Адрес массива передается первым параметром, размер массива — вторым. Вызывающая программа должна положить эти значения в стек в обратном порядке.

```

ArrZero proc
    Param_Arr equ [BP + 4]
    Param_Size equ [BP + 6]

    push BP
    mov BP, SP

    push AX
    push BX
    push CX

    xor AX, AX
    mov BX, Param_Arr
    mov CX, Param_Size
11:
    mov [BX], AX
    inc BX
    loop 11

    pop CX
    pop BX
    pop AX

    pop BP
    ret 2 * 2
ArrZero endp

```

```

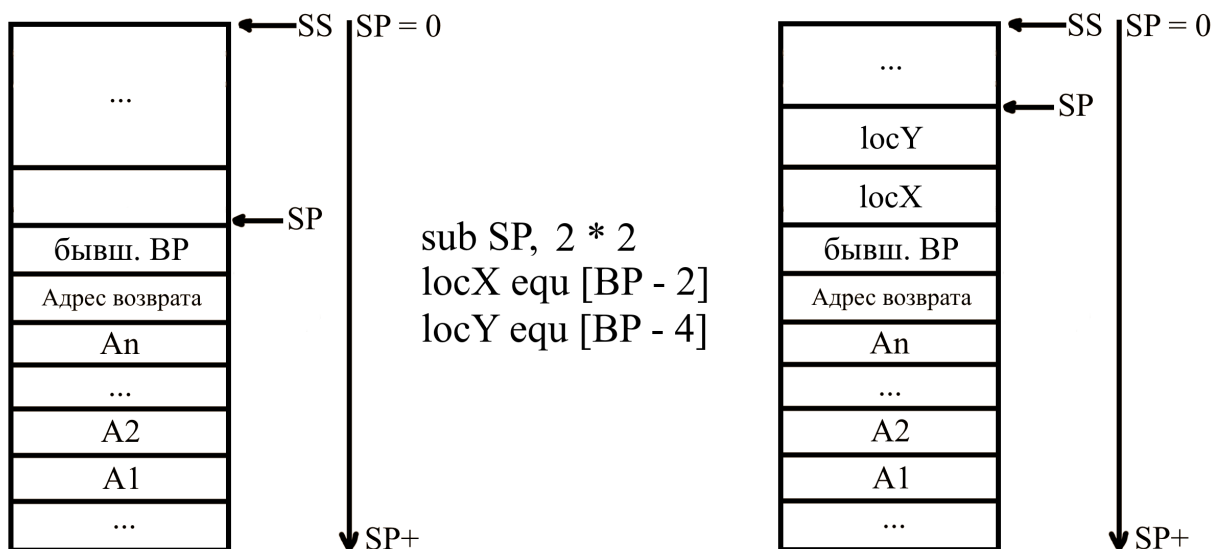
; ВЫЗОВ:
mov AX, ArrSize
push AX
lea AX, Arr
push AX
call ArrZero

```

6. Локальные параметры в процедуре.

Локальные параметры процедуры можно хранить в стеке. Для этого в начале процедуры нужно отнять от SP то количество байт, которое потребуется для хранения локальных параметров.

Предварительно сохранив значение SP в BP, к локальным параметрам можно будет обращаться при помощи выражений вида $[BP - n]$.



До выхода из процедуры нужно вернуть SP прежнее значение.

```

Func_With_Params proc
    push BP
    mov BP, SP
    sub SP, N    ; выделить N байт под локальные параметры

    ; ...

    mov SP, BP  ; вернуть старое значение SP, очистка
                ; локальных параметров
    pop BP

```

```
        ret K          ; очистка фактических параметров
Func_With_Params endp
```

Процесс создания локальных переменных можно упростить при помощи макро-определения `local`:

```
Func_With_Params proc
    local locX :word, locY :word
        push BP
        mov BP, SP

        ; ...

    pop BP
    ret K          ; очистка фактических параметров
Func_With_Params endp
```

7. Организация работы со строками, установка и сброс флагов.

💡 Везде, где упоминаются регистры SI и DI, могут также использоваться ESI или EDI.

Строка — последовательность байтов, слов или двойных слов.

Все команды работы со строками подразумевают, что **источник** находится по адресу DS:SI, а **приемник** по адресу ES:DI.

Команды выполняют операцию с одной единицей строки — байтом, словом или двойным словом — в зависимости от команды. Чтобы произвести действие над всей строкой, слева от команды через пробел записывается **префикс**.

Префикс — это команда повторения операции со строками. Префиксы работают аналогично командам семейства `loop`; отнимают от CX единицу на каждой итерации, и возможно проверяют флаг ZF:

- `rep` — повторять, пока CX $\neq 0$.
- `repz` | `repz` — повторять, пока ZF = 1 и CX $\neq 0$.
- `repne` | `repnz` — повторять, пока ZF = 0 и CX $\neq 0$.

В зависимости от значения **флага** DF, команды обработки строк будут работать в сторону увеличения (DF = 0) или в сторону уменьшения (DF = 1) адресов, соответственно, прибавляя или отнимая от SI и DI какое-то значение после совершения основной операции.

Это все, на что влияет DF. Значение DF никак не отражается на порядке считывания байтов в машинных словах или двойных словах (прямой или обратный). Он только управляет *направлением обхода строк*.

Управление флагом DF:

- `std` (*SeT Direction flag*) — установить $DF = 1$;
- `cld` (*CLear Direction flag*) — сбросить $DF = 0$.

Также могут быть полезны следующие команды для манипуляции `FLAGS`:

- `stX` (*SeTX*) — установить флаг X . X — это первая буква какого-нибудь флага, например, `stc` установит флаг $CF = 1$.
- `clX` (*CLear X*) — сбросить флаг X .
- `lahf` — скопировать младший байт регистра `FLAGS` в `AH`.
- `sahf` — скопировать из `AH` в младший байт регистра `FLAGS` (включает флаги `SF`, `ZF`, `AF`, `PF`, `CF`).

8. Команды для работы со строками: `movs`, `lods`, `stos`.

💡 Везде, где упоминаются регистры `SI` и `DI`, могут также использоваться `ESI` или `EDI`.

Команды `movs`, как и другие команды работы со строками, всегда имеют два операнда: `DS:SI` — **источник** и `ES:DI` — **приемник**. Вне зависимости от формы команды, всегда копирует единицу памяти из источника в приемник, после чего изменяет значение `SI` и `DI`, опираясь на флаг `DF`.

Формы команды `movs`:

- `movsb` — переслать байт данных из `DS:SI` в `ES:DI` (потом изменить `SI` и `DI` на 1).
- `movsw` — переслать машинное слово из `DS:SI` в `ES:DI` (потом изменить `SI` и `DI` на 2).
- `movsd` — переслать двойное слово из `DS:SI` в `ES:DI` (потом изменить `SI` и `DI` на 4).
- `movs M, M`; эту форму команды лучше не использовать, так как в этом нет большой необходимости. Пересылает байт, машинное слово или двойное слово из `DS:SI` в `ES:DI`, в зависимости от типа операндов, потом изменяет `SI` и `DI`. Значение операндов неважно, важен их размер (который должен быть одинаковым).

Ей можно найти только одно применение: при желании, можно переопределить селектор `DS` в операнде `DS:SI` на какой-нибудь другой селектор. Например, `movs byte ptr [FS:BX], byte ptr [ES:DI]` изменит первый операнд команды на `FS:SI`. Однако команда `movs` не позволяет никоим образом изменить операнд `ES:DI`.

Команды `lods` загружают байт, слово или двойное слово из памяти по адресу `DS:SI` в регистр `AL`, `AX` или `EAX`, соответственно (и изменяют значение `SI`).

- `lodsb;`
- `lodsw;`
- `lodsd;`
- `lods M;` опять же, лучше не использовать. Пересылает единицу памяти в регистр, размер данных зависит от размера операнда. После пересылки изменяет значение `SI`. Подобно `movs M, M`, данная команда позволяет переопределить селектор в источнике `DS:SI` на другой селектор.

Команды `stos` делают противоположное тому, что делали `lods`: они загружают значение из регистра (`AL`, `AX` или `EAX`) в область памяти по адресу `ES:DI`, после чего изменяют значение `DI`:

- `stosb;`
- `stosw;`
- `stosd;`
- `stos M;` нет совсем никакого смысла использовать, так как аналогично команде `movs M, M`, нельзя переопределить селектор в приемнике `ES:DI`.

9. Команды для работы со строками: `cmps`, `scas`, `ins`, `outs`.

💡 Везде, где упоминаются регистры `SI` и `DI`, могут также использоваться `ESI` или `EDI`.

Команды `cmps` выполняют сравнение (как `cmp`) байтов, слов или двойных слов по адресам `DS:SI` и `ES:DI` и устанавливают нужные флаги соответственно. Аналогично командам `movs`, после основной операции `SI` и `DI` увеличиваются или уменьшаются в зависимости от флага `DF`:

- `cmpsb;`
- `cmpsw;`
- `cmpsd;`

- `cmps M, M`. Подобно `movs M, M`, сравнивает байт, слово или двойное слово в зависимости от размера предоставленных явных операндов (который конечно же должен быть одинаковым). После операции изменяет регистры `SI` и `DI`. Абсолютно также, как и с `movs M, M`, лучше избегать использования этой формы, но в случае, если надо переопределить селектор `DS` на какой-нибудь другой селектор, то эта форма позволяет такое сделать.

`cmps` можно использовать с префиксами

- `repz` | `repz`, чтобы сравнение выполнялось до первого несовпадения;
- `repne` | `repnz`, чтобы сравнение выполнялось до первого совпадения.

Команды `scas` сравнивают значение регистра `AL`, `AX` или `EAX` со значением в области памяти по адресу `ES:DI`, устанавливают флаги, как после `cmp`. После сравнения `DI` увеличивается или уменьшается:

- `scasb`;
- `scasw`;
- `scasd`;
- `scas M`. В зависимости от размера операнда `M`, сравнивает значение в регистре со значением в памяти по адресу `ES:DI`, после чего изменяет `DI` в соответствии с флагом `DF`.

Нет никакого смысла использовать, так как, подобно другим командам, селектор `ES` никоим образом заменить нельзя.

Команды `ins` считывают байт, слово или двойное слово из порта ввода/вывода, номер которого расположен в регистре `DX`, результат считывания записывается в область памяти по адресу `ES:DI`:

- `insb`;
- `insw`;
- `insd`;
- `ins M, DX`. Ничего нового, в зависимости от размера `M` считывает байт, слово или двойное слово из порта с номером `DX` в память по адресу `ES:DI`. Так же лучше избегать этой формы, потому что никакого преимущества у нее нет (`ES` заменить нельзя).

И наконец, команды `outs` записывают в порт ввода/вывода с номером `DX` значение в памяти, находящееся по адресу `DS:SI`:

- `outsb`;

- outsw;
- outsd;
- outs DX, M.

История повторяется ... Если очень надо заменить селектор DS на какой-то другой, то это можно сделать такой формой команды.

Пример использования команд. Простая пересылка одной области памяти в другую:

```

; ...
.data
X dw 100 dup (?)
Y dw 100 dup (?)

Addr_X dd X
Addr_Y dd Y
.code
; ...
cld                      ; DF = 0
lds SI, Addr_X           ; DS = @data, SI = offset X
les DI, Addr_Y           ; ES = @data, DI = offset Y
mov CX, 100              ; количество машинных слов
rep movsw                ; переслать 100 машинных слов
; ...

```

В строке S заменить первое вхождение символа '*' на '.':

```

; ...
.data
S db 500 dup (?)
.code
; ... инициализация S ...
cld                      ; DF = 0
mov AX, @data
mov DS, AX               ; DS = @data
push DS
pop ES                   ; ES = DS = @data
lea DI, S                ; DI = offset S

mov CX, 500
mov AL, '*'
repne scasb              ; до первой '*' или до конца строки
jne finish
mov byte ptr ES:[DI - 1], '.' ; заменить
finish:                  ; '*' не найдена
; ...

```

10. Организация работы со строками переменной длины в Ассемблере.

Можно подойти к этой проблеме с двух сторон:

- как в C/C++, оканчивать строки специальным символом-терминатором. Этот способ позволяет создавать строки неограниченной длины, но чтобы найти конец строки, зная адрес ее начала, придется сделать N сравнений.
- как в Pascal, задать максимально возможный размер строки (например, 255 байт), и первый байт всех строк использовать для хранения ее текущей длины. Тогда поиск конца строк будет происходить за линейное время.

Пример работы с «паскалевским» вариантом строк. Нужно найти в строке S символ '*' и удалить его из строки:

```
        ; где-то в сегменте описана строка S, первым
        ; байтом хранящая свой размер
mov AX, @data
mov DS, AX          ; DS = @data
push DS
pop ES              ; ES = DS

cld                ; DF = 1
lea DI, S + 1       ; DI = (offset S) + 1
xor CH, CH
mov CL, S           ; длину строки в CX
mov AL, '*'
repne scasb         ; поиск '*'
jne finish

        ; звездочка найдена на i-й позиции
mov SI, DI          ; SI = i + 1
dec DI              ; DI = i
rep movsb
dec S               ; удалили символ, размер строки уменьшился

finish:
        ; ...
```

11. Блоки повторений и макрооператоры в Ассемблере.

Макросредства позволяют генерировать, модифицировать текст программы на Ассемблере в процессе трансляции.

К макросредствам относят: блоки повторений, макросы, директивы условной генерации.

Программы, написанные на макроязыке, транслируются в два этапа. Сначала все макросредства разворачиваются в чистый Ассемблер, этот этап называют **макрогенерацией** (препроцессинг). Затем выполняется **ассемблирование** — перевод в машинный код.

Блоки повторения в процессе макрогенерации заменяются указанной последовательностью команд столько раз, сколько задано в заголовке блока, причем содержимое блока может модифицироваться.

Макросы похожи на подпрограммы. Описание макроса называется *макроопределением*, обращение — *макрокомандой*. Результат *макроподстановки* — *макрорасширение*. Макросы лучше использовать для маленьких подпрограмм, а процедуры для объемных.

Общий вид **блока повторений**:

```
<заголовок>
    <тело>
endm
```

<заголовок> может быть:

- `rept n`, где `n` — константное выражение. В результате `n` копий тела блока повторения записывается на его месте.

Пример создания массива из 256 элементов, заполненный числами от 0 до 255:

```
n = 1
Arr db 0
rept 255
    db n
    n = n + 1
endm

; Arr db 0
;     db 1
;     db 2
;     ...
;     db 255
```

- `irp p, <v1, v2, ..., vn>`. `p` — имя формального параметра, который является локальным в теле блока повторения. Тело блока повторится `n` раз, последовательно подставляя в тело вместо `p` `v1, v2, ..., vn`.

Пример отправки указанных регистров в стек одним блоком повторений:

```
irp R, <AX, BX, CX, SI>
    push R
endm
```

```
; push AX
; push BX
; push CX
; push SI
```

Причем стоит помнить, что это всего лишь текстовые замены. То есть не обязательно, чтобы в < > были перечислены какие-то объекты.

- `irpc p, <s1s2...sn>`

Работает примерно так же, как и `irp`, только последовательно подставляет не строки, а символы `s1, s2, ..., sn`.

< > можно не использовать, если внутри них не используется пробел или точка с запятой.

Пример:

```
irpc param, 175p
    add AX, param
endm
```

```
; add AX, 1
; add AX, 7
; add AX, 5
; add AX, p
```

В макроопределениях и в блоках повторения могут использоваться специальные операторы Ассемблера, называемые **макрооператорами**.

1. Оператор `&` — указывает границы формального параметра, выделяет его из окружающего текста, при транслировании в текст программы не записывается. Примеры:

```
irp k, <1, 5, 7>
    var&k dw ?
endm
```

```
; var1 dw ?
; var5 dw ?
; var7 dw ?
```

```

irpc A, "<
        db 'A, &A, &A&B'
endm

; db 'A, ", "B'
; db 'A, <, <B'

```

Если рядом стоит несколько знаков &, макрогенератор удаляет за проход только один из них:

```

irpc P1, AB
    irpc P2, HL
        inc P1&&P2
    endm
endm

; После первого прохода:
; irpc P2, HL
;     inc A&P2
; endm
; irpc P2, HL
;     inc B&P2
; endm

; После второго прохода:
; inc AH
; inc AL
; inc BH
; inc BL

```

- Оператор <> — позволяет передать операнд с запятыми, пробелами и точкой с запятой, как одну цельную строку:

```

irp v, <<1, 2>, 3>
    db v
endm

; db 1, 2
; db 3

```

```

irpc s, <A; B>
    db 's'
endm

; db 'A'

```

```
; db ';'
; db 'B'
```

3. Оператор `!` — действует, как `<>`, только на один единственный символ, идущий после него.
4. Оператор `%` — указывает на то, что следующий текст является константным выражением, которое должно быть вычислено перед подстановкой:

```
k equ 4
irp A, <k + 1, %k + 1, W%k + 1>
    dw A
endm

; k equ 4
; dw k + 1
; dw 5
; dw w5
```

5. Оператор `;;` — начало макрокомментария.

12. Директивы условной генерации и их назначение.

Директивы условного ассемблирования управляют процессом ассемблирования путем подключения или отключения фрагментов исходного текста программы.

Общий вид таких директив:

```
if выражение
    if-часть
[ elseif выражение
    elseif-часть ]
[ else
    else-часть ]
endif
```

У `if` и `elseif` может быть много вариаций, ПОЭТОМУ [давайте притворимся, что стало немно-го] рассмотрим некоторые из них:

1. `if константное_выражение`
`elseif константное_выражение`
`if-часть` ассемблируется, если `константное_выражение` не равно 0.
2. `ife константное_выражение`
`elseif константное_выражение`
`if-часть` ассемблируется, если `константное_выражение` равно 0.

3. `ifdef метка`
`elseifdef метка`
if-часть ассемблируется, если метка определена.
4. `ifndef метка`
`elseifndef метка`
if-часть ассемблируется, если метка не определена.
5. `ifb <аргумент>`
`elseifb <аргумент>`
if-часть ассемблируется, если значением аргумента является пробел.
6. `ifnb <аргумент>`
`elseifnb <аргумент>`
if-часть ассемблируется, если значением аргумента не является пробел.
7. `ifdif <арг. 1>, <арг. 2>`
`elseifdif <арг. 1>, <арг. 2>`
if-часть ассемблируется, если <арг. 1> отличен от <арг. 2>. Регистр учитывается.
8. `ifdifi <арг. 1>, <арг. 2>`
`elseifdifi <арг. 1>, <арг. 2>`
if-часть ассемблируется, если <арг. 1> отличен от <арг. 2>. Регистр не учитывается.
9. `ifidn <арг. 1>, <арг. 2>`
`elseifidn <арг. 1>, <арг. 2>`
if-часть ассемблируется, если <арг. 1> и <арг. 2> одинаковые. Регистр учитывается.
10. `ifidni <арг. 1>, <арг. 2>`
`elseifidni <арг. 1>, <арг. 2>`
if-часть ассемблируется, если <арг. 1> и <арг. 2> одинаковые. Регистр не учитывается.

В списке там, где встречаются угловые скобки, они требуются обязательно.

13. Макросы в Ассемблере, их описание и использование.

Общий вид **макроопределения**:

```
<имя макроса> масро <формальные параметры>  
    local <список имен>  
    <тело>  
endm
```

<имя макроса> будет использоваться при обращении к нему. <формальные параметры> записываются через запятую, это набор локальных имен. Директива `local` <список имен> перечисляет через запятую имена меток, которые будут локальными для одного макрорасширения.

Макрокоманда — обращение к макросу:

<имя макроса> <фактические параметры>

Фактические параметры указываются через запятую или через пробел. Фактический параметр может быть заключен в кавычки или <>, если он содержит запятые, пробелы или точки с запятой.

С помощью `exitm` можно досрочно выйти из макроса.

С помощью `purge` <имя макроса> можно удалить созданное макроопределение для оставшейся программы.

Макрос `if x < y then goto L:`

```
If_Less_Then_L macro x, y, L
    mov AX, x
    cmp AX, y
    jl L
endm

; Использование: поиск минимального из 3 чисел
; DX = min(A, B, C)
mov DX, A
If_Less_Then_L A, B, M1
mov DX, B
M1:
    If_Less_Then_L DX, C, M2
    mov DX, C
M2:
    ; ...
```

С помощью макросов можно упростить обращение к процедурам:

```
GCD macro x, y
    mov AX, x
    mov BX, y
    call GCD_Func    ; AX = НОД(x, y)
endm

; Вычислить CX = НОД(A, B) + НОД(C, D)
GCD A, B            ; AX = НОД(A, B)
mov CX, AX           ; CX = НОД(A, B)
GCD C, D            ; AX = НОД(C, D)
add CX, AX           ; CX = НОД(A, B) + НОД(C, D)
```

14. Примеры использования директив условной генерации.

Пример использования ДУА — включение/отключение отладочного вывода программы:

```
DEBUG equ 1           ; отладка

    mov x, AX
    if DEBUG
        PrintInt x
    endif
    mov BX, 0
```

Другие примеры использования ДУА:

```
; Очень плохой пример, но он был в презентации,
; поэтому лучше ознакомиться с ним, чем нет.

; Макрос, выполняющий операцию логического сдвига значения
; переменной x на n разрядов вправо.
Shift macro x, n
    ife n - 1
        shr x, 1
    else
        mov CL, n
        shr x, CL
    endif
endm

    shift A, 5           ; mov CL, 5
                        ; shr A, CL
```

```
; Макрос, устанавливающий переменную x в значение 0.
Set_0 macro x
    if type x eq dword
        mov dword ptr x, 0
    elseif type x eq word
        mov word ptr x, 0
    else
        mov byte ptr x, 0
    endif
endm
```

15. Логические выражения в директивах условной генерации.

*; Макрос, выполняющий сдвиг байтового значения
; x на n бит вправо, если $0 < n < 8$.*

```
RShiftByte macro x, n
    if (n gt 0) and (n lt 8)
        mov CL, n
        shr x, CL
    elseif n ge 8
        mov x, byte ptr 0
    endif
endm
```

*; Вычисляет минимум или максимум из двух величин,
; помещает результат в R1. Имя операции
; передается в T.*

```
Max_Min macro R1, R2, T
    local L
    ifdif <R1>, <R2>      ;; R1 и R2 - разные регистры
        cmp R1, R2
        ifidni <T>, <max> ;; T == max
            jge L
        else              ;; T == min
            jle L
        endif
L:
    mov R1, R2
    endif
endm
```